

ESP-IDF Extension for VSCode

[ESP-IDF Extension for VSCode](#)

- »
 - ESP-IDF Extension for VSCode
 - [Edit on GitHub](#)
-

ESP-IDF Extension for VSCode

[\[ä.æ](#)

The ESP-IDF extension for VSCode Code empowers developers to efficiently develop, build, flash, monitor, debug, and manage projects targeting Espressif chips using the Espressif IoT Development Framework (ESP-IDF). This extension integrates seamlessly with Visual Studio Code, providing a familiar environment for streamlined development workflows. Whether you are a beginner or an experienced developer, this documentation will guide you through setting up, configuring, and utilizing the ESP-IDF extension to leverage the full potential of Espressif chips in your IoT applications.

- [Prerequisites](#)
- [Install ESP-IDF and Tools](#)
- [Start a Project](#)
- [Connect Your Device](#)
- [Configure Your Project](#)
- [Build Your Project](#)
- [Flash onto the Device](#)
- [Monitor the Output](#)
- [Debug Your Project](#)
- [Additional IDE Features](#)
- [Troubleshooting](#)
- [Settings](#)
- [See all Commands](#)
- [Extension Activation](#)
- [FAQs](#)
- [Release Notes](#)

Features

Application level tracing

Build your project

CMakeLists editor

Code coverage

Debug your device

Doctor command troubleshooting

eFuse viewer

Flash a specific partition or get binaries from the current device partition tree

Flash your device using UART, DFU (ESP32-S2 and ESP32-S3), or JTAG (using OpenOCD)

Heap tracing

Install ESP components from ESP Component Registry and use Arduino as an ESP-IDF component

Manage multiple configurations for the same project

Manage multiple projects in the same window

Monitor the output of your device

NVS partition editor

New project wizard

Partition editor

Postmortem debug with core dump or GDB stub

Select serial port

Setup wizard

Size analysis of binaries

System view tracing

Unit testing

Use QEMU to emulate debug and monitor output

Use the extension in a Docker container

Use the extension in WSL

For more about this project, please see <https://github.com/espressif/vscode-esp-idf-extension>.

[Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).

- [Download PDF](#)

Prerequisites

[ESP-IDF Extension for VSCode](#)

- »
- Prerequisites
- [Edit on GitHub](#)

Prerequisites

[\[ä,æ!\[\]\(e78f798d4ea5c530c9db49e7d26e6b95_img.jpg\)](#)

[Visual Studio Code](#)

[Espressif Install Manager prerequisites](#)

Note

No additional prerequisites are required. The [installation](#) step will handle the rest.

Next, proceed to [Install ESP-IDF and Tools](#).

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Install ESP-IDF and Tools

[ESP-IDF Extension for VSCode](#)

- »
 - Install ESP-IDF and Tools
 - [Edit on GitHub](#)
-

Install ESP-IDF and Tools

[\[æ\]](#)

After installing Visual Studio Code (VS Code), install the ESP-IDF extension for VS Code.

Navigate to `View > Extensions` or the keyboard shortcut `Ctrl+Shift+X` in Windows/Linux or `Shift+⌘+X` in macOS.

Search for [ESP-IDF](#) in the list of extensions.

Install the ESP-IDF extension.

Navigate to `View > Command Palette`.

Type ESP-IDF: Open ESP-IDF Installation Manager to download and run the ESP-IDF Installation Manager to install the ESP-IDF framework. A loading notification will appear, followed by the execution of the installer.

Note

For versions of ESP-IDF < 5.0, spaces are not supported in configured paths.

Alternatively, you can download the ESP-IDF Installation Manager from the following link [ESP-IDF Installation Manager](#) among the following options:

Download: Faster speed in China using Espressif download servers.

GitHub: Using GitHub release links.

Use the ESP-IDF Installation Manager to install the ESP-IDF and tools. If necessary, here is the [ESP-IDF Installation Manager Documentation](#).

Note

In remote or headless environments such as SSH, WSL, Dev Containers, Codespaces, or browser-based VS Code, the extension runs `eim wizard` instead of the GUI. When launching the

wizard on Linux, the extension also adds the EIM executable directory to the remote user's shell PATH so `eim` can be run directly in future terminals. See the [EIM CLI Commands](#) reference for the available commands.

All ESP-IDF versions installed using the ESP-IDF Installation Manager will be automatically detected by the ESP-IDF VS Code extension by reading EIM's `eim_idf.json` file.

Note

The default `eim_idf.json` file is located at: `C:\Espressif\tools\eim_idf.json` for Windows and `$HOME/.espressif/tools/eim_idf.json` for macOS/Linux. If your `eim_idf.json` is not in the default location, you can define the path to the EIM `eim_idf.json` file using the `idf.eimIdfJsonPath` extension configuration setting in Visual Studio Code Preferences: Open Settings (UI) command.

In Visual Studio Code, navigate to `View > Command Palette` and type `select current esp-idf version` and select **ESP-IDF: Select Current ESP-IDF Version** from the list.

The list of available ESP-IDF setups will be shown, select which one you want to use for the current ESP-IDF project.

The selected setup will save a `idf.currentSetup` with selected ESP-IDF path and the extension will configure required environment variables for the current ESP-IDF project saved as workspace folder state.

You can review the setup by running the **ESP-IDF: Doctor Command** by navigating to `View > Command Palette` and type `doctor command` and select **ESP-IDF: Doctor Command** from the list.

The next step is to [Create an ESP-IDF Project](#).

Warning

Check the [Troubleshooting](#) section if you encounter any issues during installation.

Manual configuration of ESP-IDF and Tools in the ESP-IDF extension for VS Code

Note

If you configure the extension using environment variables, the extension will use it over any selected ESP-IDF setup in `idf.currentSetup`. So make sure to clear the environment variables if you want to use the selected ESP-IDF setup. If you use the `ESP-IDF: Select Current ESP-IDF Version` command to select an ESP-IDF setup, the extension will use the environment variables from the selected IDF setup and delete manually configured environment variables `IDF_PATH`, `IDF_TOOLS_PATH` and `IDF_PYTHON_ENV_PATH`.

You can manually configure the ESP-IDF extension for VS Code to use your existing ESP-IDF setup by setting required environment variables for the ESP-IDF extension in Visual Studio Code settings. You would need the ESP-IDF path (IDF_PATH), the set of ESP-IDF Tools to be appended in PATH or the ESP-IDF tools path (IDF_TOOLS_PATH), and Python environment path (IDF_PYTHON_ENV_PATH) to configure the extension.

For example, if you have ESP-IDF installed at `/home/user/esp-idf`, and the tools are located at `/home/user/.espressif/`, and your Python virtual environment is located at `/home/user/.espressif/python_env/idf6.1_py3.13_env`, you can configure the extension as follows:

Open Command Palette (press shortcut F1) and type `Preferences: Open Settings (JSON)` and select the command.

Add the following configuration to the `settings.json` file, replacing the paths with the actual paths to your ESP-IDF and tools:

```
{
  "idf.customExtraVars": {
    "IDF_PATH": "/home/user/esp-idf",
    "IDF_TOOLS_PATH": "/home/user/.espressif",
    "IDF_PYTHON_ENV_PATH": "/home/user/.espressif/python_env/idf6.1_py3.13_env"
  }
}
```

Or alternatively you can use `PATH` (or `Path` in Windows) with list of tools to be appended to `PATH`:

```
{
  "idf.customExtraVars": {
    "IDF_PATH": "/home/user/esp-idf",
    "PATH": "/home/user/.espressif/tools/xtensa-esp-elf-gdb/16.3_20250913/xtensa-esp-elf-gdb/bin:/h",
    "IDF_PYTHON_ENV_PATH": "/home/user/.espressif/python_env/idf6.1_py3.13_env"
  }
}
```

If you are using an ESP-IDF setup not installed using ESP-IDF Installation Manager (EIM), you can use the following command to get these environment variables from your ESP-IDF setup:

```
source /home/user/esp-idf/export.sh
python /home/user/esp-idf/tools/idf_tools.py export --format key-value
```

If you installed ESP-IDF using the ESP-IDF Installation Manager (EIM) and you want to use the ESP-IDF activation script to get required environment variables, you can use the following command:

```
/home/user/.espressif/tools/activate_idf_v6.0.sh -e
```

Uninstall ESP-IDF VS Code Extension

To uninstall the ESP-IDF VS Code extension, follow these steps:

Open Command Palette (press shortcut F1) and type `ESP-IDF: Remove ESP-IDF Settings`. Select the command to remove all ESP-IDF settings.

Navigate to `View > Extensions` or use the keyboard shortcut `Ctrl+Shift+X` in Windows/Linux or `Shift+⌘+X` in macOS.

Search for ESP-IDF and click the `Uninstall` button.

Remove the following folders:

Go to your `${VSCODE_EXTENSION_DIR}` and delete the ESP-IDF plugin folder.

`${VSCODE_EXTENSION_DIR}` is the location of the extension:

Windows: `%USERPROFILE%\.vscode/extensions/espressif.esp-idf-extension-VERSION/`

macOS/Linux: `$HOME/.vscode/extensions/espressif.esp-idf-extension-VERSION/`

Note

Make sure to replace `VERSION` with the actual version number of the installed ESP-IDF extension.

FAQ

How do I know if the ESP-IDF extension is installed correctly?

Open Command Palette (press shortcut F1) and type `ESP-IDF: Doctor Command`. Select the command to run the doctor command, which will check the installation and configuration of the ESP-IDF extension and tools.

When I use the `ESP-IDF: Select Current ESP-IDF Version` command, it shows no ESP-IDF setup available. What should I do?

Make sure you have installed the ESP-IDF using the ESP-IDF Installation Manager (EIM) or manually configured the extension with the correct environment variables. If you have installed ESP-IDF using EIM, ensure that the **`eim_idf.json`** file is in the default location or set the path to it in the extension settings `idf.eimIdfJsonPath`.

The `eim_idf.json` exists and the extension does not detect any ESP-IDF setups. What should I do?

Ensure that the **`eim_idf.json`** file is correctly formatted and contains valid paths to your ESP-IDF installations. Check that EIM installed ESP-IDF correctly.

EIM didn't install ESP-IDF correctly, what should I do?

Check the EIM logs for any errors during installation. You can find the logs in the [EIM logs directory](#). If you encounter issues, try reinstalling ESP-IDF using EIM or manually configuring the extension with the correct environment variables.

I have multiple ESP-IDF versions installed, how do I switch between them?

Use the ESP-IDF: Select Current ESP-IDF Version command in the Command Palette to select the desired ESP-IDF version for your current project.

I have manually configured the extension with environment variables, but I want to switch to using a selected ESP-IDF setup. How do I do that?

Clear the manually configured environment variables in the extension settings and use the ESP-IDF: Select Current ESP-IDF Version command to select the desired ESP-IDF setup.

I need help with a specific issue, where can I get support?

You can seek help from the [EIM GitHub](#) or the [ESP-IDF Extension for VS Code GitHub](#).

I'm using AI tools to troubleshoot my ESP-IDF installation, but it's not working. What should I do?

Make sure to add the [Espressif Documentation MCP](#) to your AI tool's knowledge base. You can alternatively just add documentation links to [EIM](#) and [ESP-IDF Extension for VS Code](#).

This extension also contributes MCP Servers to the current IDE being used.

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Start a Project

[ESP-IDF Extension for VSCode](#)

- »
 - Create an ESP-IDF Project
 - [Edit on GitHub](#)
-

Create an ESP-IDF Project

[\[ä.æ!\[\]\(8b57f0e15e7dda24cf9977561475f640_img.jpg\)](#)

You can start a project in three ways:

[Using ESP-IDF: New Project](#)

[Opening an Existing ESP-IDF Project](#)

[ESP Component Registry](#)

The first option is recommended because it allows to configure the project. The second and third options create the project with the current workspace folder configuration.

Using [ESP-IDF: New Project](#)

In Visual Studio Code:

Navigate to View > Command Palette.

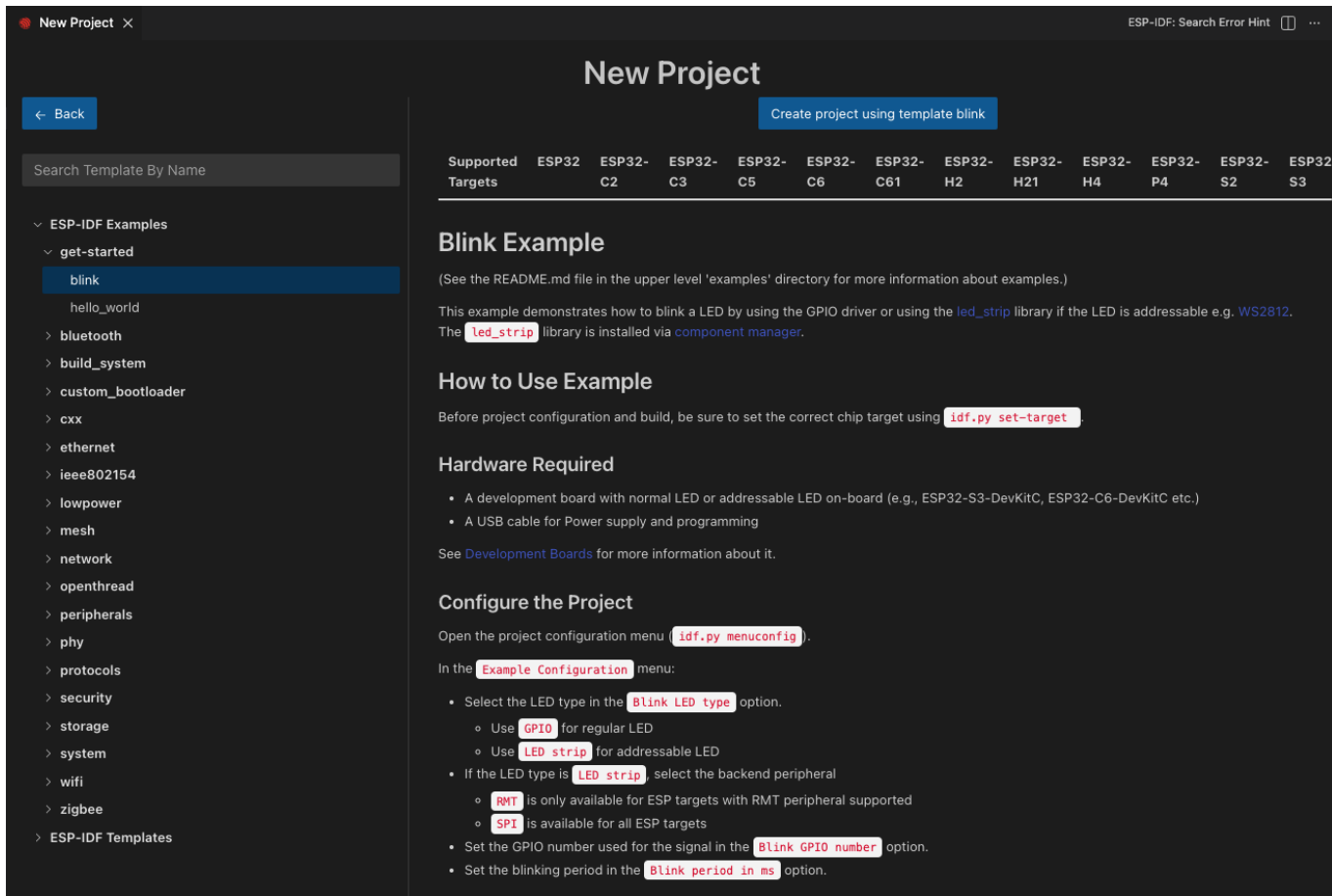
Type ESP-IDF: New Project and select the command to launch the New Project wizard.

A dropdown menu appears with all ESP-IDF setups detected by the extension. Select the desired ESP-IDF setup to use for the new project creation.

A menu with all examples from the ESP-IDF is displayed. You can select one of the examples as a template for your new project.

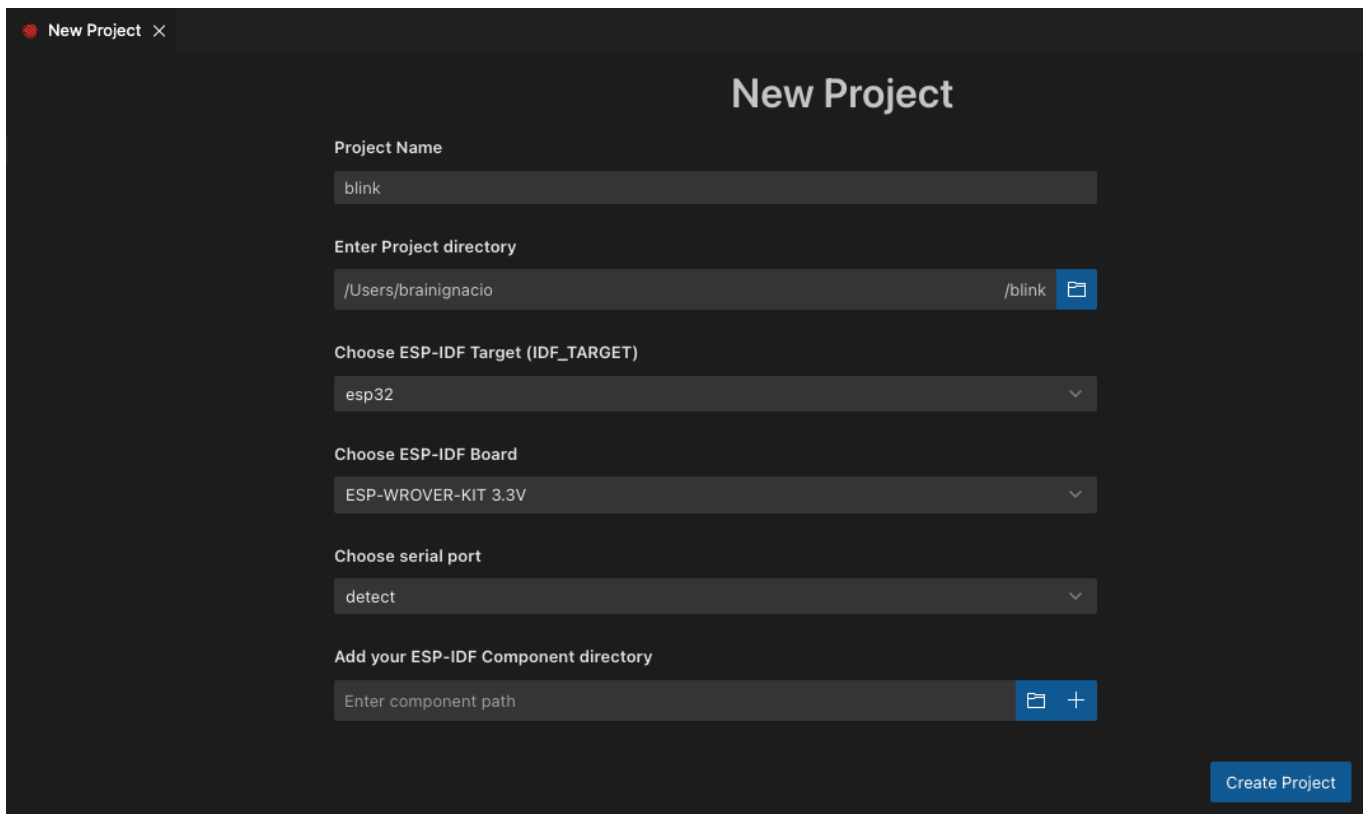
Note

If you want to create a blank project, choose ESP-IDF `sample_project` or Extension `template-app`.



Choose your desired template and click the Create Project Using Template <template-name> button, where <template-name> is the name of the selected template.

The New Project configuration window appears. Fill in the required fields:



Choose the project name.

Choose the location for the new project.

Select the Espressif IDF_TARGET and Board to use in this new project.

Select the serial port of the device (a list of currently connected serial devices appears in the dropdown).

Note

If you are not sure about the serial port name, refer to [Establish Serial Communication](#).

Note

Please refer to [Configuration of OpenOCD for Specific Target](#) to select the appropriate OpenOCD configuration file based on your hardware.

Optionally, you can import any ESP-IDF component directory component-dir to the new project. This will copy it to the new project's components subdirectory (<project-dir>/components/component-dir).

Click the Create Project button.

Wait for the project to be created and then click on open Project.

Opening an Existing ESP-IDF Project

ESP-IDF projects follow this directory structure:

[ESP-IDF Example Project](#)

```
- myProject/
  - CMakeLists.txt
  - sdkconfig
  - components/
    - component1/
      - CMakeLists.txt
      - Kconfig
      - src1.c
    - component2/
      - CMakeLists.txt
      - Kconfig
      - src1.c
      - include/
        - component2.h
  - main/
    - CMakeLists.txt
    - src1.c
    - src2.c
  - build/
```

In Visual Studio Code:

Navigate to View > Command Palette.

Type ESP-IDF: Import ESP-IDF Project and select the command to import an existing ESP-IDF project.

This command adds both Visual Studio Code configuration files (settings.json, launch.json) and Docker container files (Dockerfile and .devcontainer.json).

The next step is to [Connect a device](#).

Adding Visual Studio Code configuration files and Docker containeri

In Visual Studio Code, go to File > open Folder and open a directory containing a CMakeLists.txt file in the root (e.g., myProject), which follows the ESP-IDF structure.

To add Visual Studio Code configuration files (settings.json, launch.json):

Navigate to View > Command Palette.

Type ESP-IDF: Add VS Code Configuration Folder and select the command.

To open the project within the ESP-IDF Docker container:

Navigate to View > Command Palette.

Type ESP-IDF: Add Docker Container Configuration and select the command to add the .devcontainer directory to your current directory.

Navigate to View > Command Palette.

Type Dev Containers: Open Folder in Remote Container and select the command to open the existing project inside the container created from the Dockerfile in the previous step.

ESP Component Registry

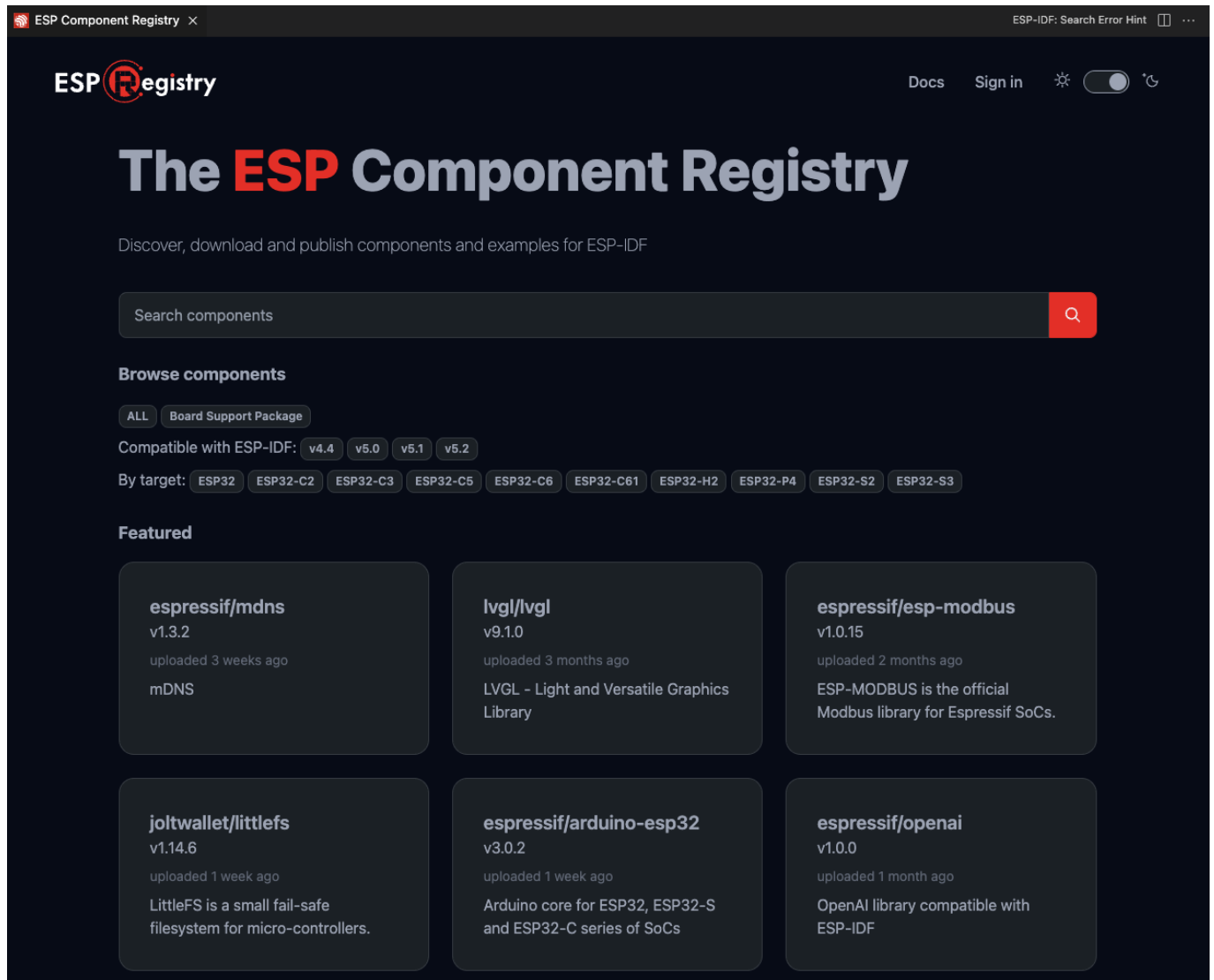
The [ESP Component Registry](#) is a collection of ESP-IDF components that can be easily added to your project. You can browse the registry, install components and create new ESP-IDF projects from component examples directly from Visual Studio Code.

In Visual Studio Code:

Navigate to View > Command Palette.

Type ESP-IDF: Show ESP Component Registry and select the command to open the ESP Component Registry UI.

The ESP-IDF: Show ESP Component Registry command launches a UI showing the [ESP Component Registry](#).



You can also create a new project from an ESP component example using the `create project from this example` button.

ESP Component Registry ×ESP-IDF: Search Error Hint

ESPRegistryDocsSign in

query_advertise

Example of the component [espressif/mdns v1.3.2](#)

mDNS example

Shows how to use mDNS to advertise and query services and hosts

Example workflow

- mDNS is initialized with host name and instance name defined through the project configuration and `_http._tcp` service is added to be advertised
- A delegated host `esp32-delegated._local` is added and another `_http._tcp` service is added for this host.
- WiFi STA is started and trying to connect to the access point defined through the project configuration
- The system event handler is used to pass the network events to mDNS so the service is aware when the interface comes up or down
- GPIO0 (BOOT Button) is initialized as pulled-up input that can be monitored for button press
- Example task is started to check if the button is pressed so it can execute the mDNS queries defined

Configure the project

- Open the project configuration menu (`idf.py menuconfig`)
- Configure Wi-Fi or Ethernet under "Example Connection Configuration" menu

Create project from this example

Or run:

```
idf.py create-project-from-example "espressif/mdns^1.3.2:query_advertise"
```

or [download archive](#) (239 bytes)

For more information, refer to [ESP Component Registry Documentation](#).

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Connect Your Device

[ESP-IDF Extension for VSCode](#)

- »
 - Connect Your Device
 - [Edit on GitHub](#)
-

Connect Your Device

[\[ä,æ](#)

In Visual Studio Code, go to View > Command Palette, enter ESP-IDF: Select Port to Use, and choose the serial port that your device is connected.

If you are not sure about the serial port name, refer to [Establish Serial Communication](#).

Next, proceed to [Configure your project](#).

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Configure Your Project

[ESP-IDF Extension for VSCode](#)

- »
 - Configure Your Project
 - [Edit on GitHub](#)
-

Configure Your Project

[\[ä.æ](#)

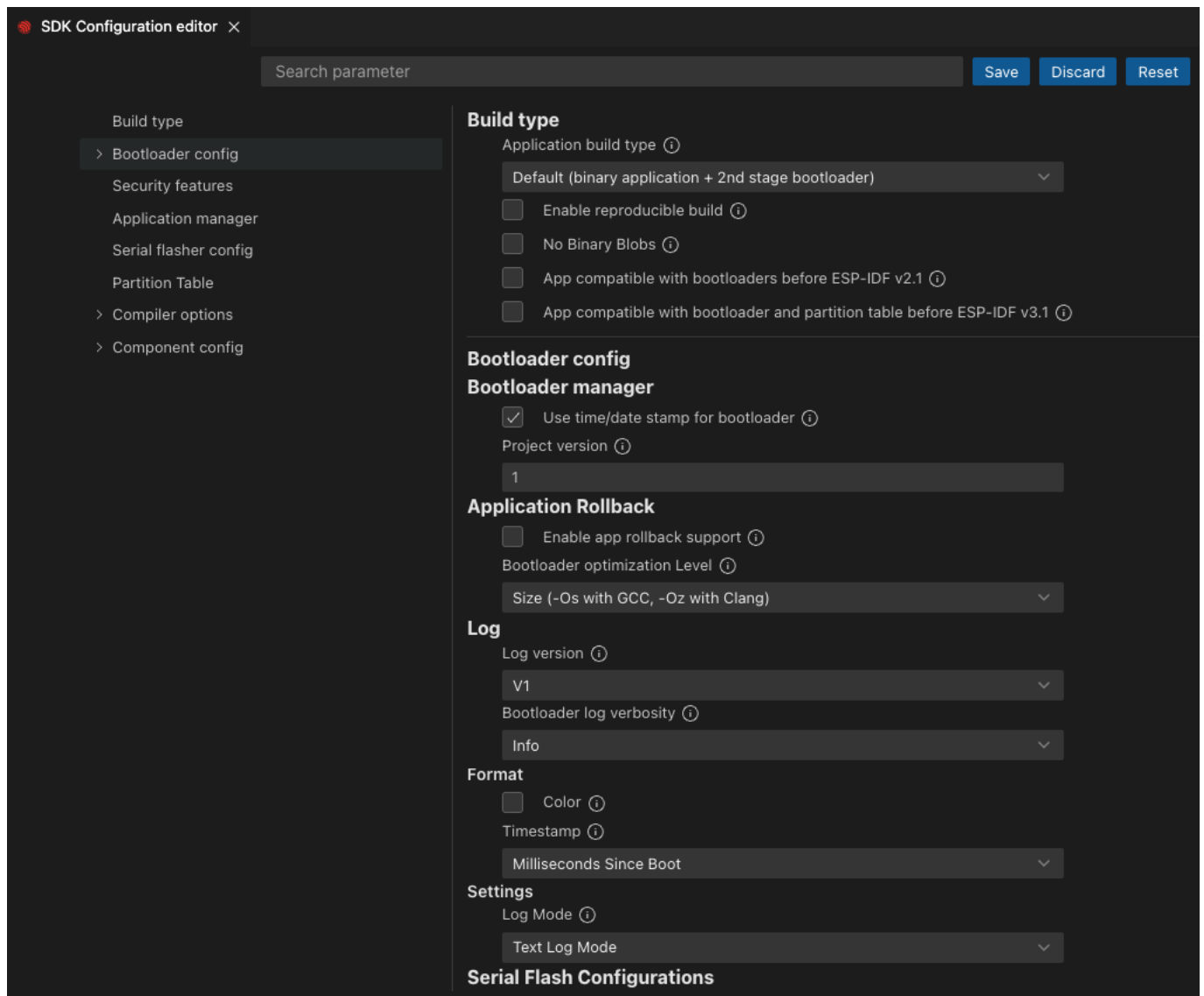
Select an Espressif target (esp32, esp32s2, etc.) by going to View > Command Palette and entering ESP-IDF: Set Espressif Device Target command.

If you are using a connected ESP-IDF development board, the OpenOCD configuration will be automatically selected based on your connected board. Otherwise, you can manually select the OpenOCD configuration by going to View > Command Palette and entering ESP-IDF: Select OpenOCD Board Configuration.

Note

Please refer to [Configuration of OpenOCD for Specific Target](#) to select the appropriate OpenOCD configuration file based on your hardware.

Next, configure your project. Go to View > Command Palette and enter ESP-IDF: SDK Configuration Editor to adjust ESP-IDF project settings.



After making changes, click save and close the window.

Next, proceed to [Build Your project](#).

Adding ESP-IDF Components

The [ESP Component Registry](#) is a collection of ESP-IDF components that can be easily added to your project. You can browse the registry, install components and create new ESP-IDF projects from component examples directly from Visual Studio Code.

In Visual Studio Code:

Navigate to View > Command Palette.

Type ESP-IDF: Show ESP Component Registry and select the command to open the ESP Component Registry UI.

The ESP-IDF: Show ESP Component Registry command launches a UI showing the [ESP Component Registry](#).

ESP Component Registry ×

ESP-IDF: Search Error Hint

ESP Registry

Docs Sign in

The ESP Component Registry

Discover, download and publish components and examples for ESP-IDF

Search components

Browse components

ALL Board Support Package

Compatible with ESP-IDF: v4.4 v5.0 v5.1 v5.2

By target: ESP32 ESP32-C2 ESP32-C3 ESP32-C5 ESP32-C6 ESP32-C61 ESP32-H2 ESP32-P4 ESP32-S2 ESP32-S3

Featured

espressif/mdns
v1.3.2
uploaded 3 weeks ago
mDNS

lvgl/lvgl
v9.1.0
uploaded 3 months ago
LVGL - Light and Versatile Graphics Library

espressif/esp-modbus
v1.0.15
uploaded 2 months ago
ESP-MODBUS is the official Modbus library for Espressif SoCs.

joltwallet/littlefs
v1.14.6
uploaded 1 week ago
LittleFS is a small fail-safe filesystem for micro-controllers.

espressif/arduino-esp32
v3.0.2
uploaded 1 week ago
Arduino core for ESP32, ESP32-S and ESP32-C series of SoCs

espressif/openai
v1.0.0
uploaded 1 month ago
OpenAI library compatible with ESP-IDF

You can browse various ESP components and install them in your current ESP-IDF project using the `install` button.

The screenshot shows the ESP Component Registry interface for the `espressif/mdns` component, version 1.3.2. The page includes a header with the ESP Registry logo, navigation links (Docs, Sign in), and a search bar. The main content area features a README section titled "mDNS Service" with a description: "mDNS is a multicast UDP service that is used to provide local network service and host discovery." It also includes an "Examples" section with a link to "Example" and a "Documentation" section with links for English and Chinese. On the right side, there are sections for "Links" (Homepage, Repository), "Supports all targets", "License: Apache-2.0", "Stats" (Archive size: 117.22 KB, Downloaded in total 1.3M times, This version: 28.0k times), and a "Badge" section showing the latest version 1.3.2. A terminal snippet shows the command: `idf.py add-dependency "espressif/mdns^1.3.2"`. The footer contains copyright information (© 2020-2024 Espressif Systems) and links for Cookie Policy, Terms of use, and Privacy Policy.

For more information, refer to [ESP Component Registry Documentation](#).

Using other ESP solutions

If you are working with ESP solutions such as ESP-Matter or ESP-RainMaker, chances are you will find them in the ESP Component Registry and you can either create projects from their examples or install the component in your current ESP-IDF project.

In case you want to use the main branch of these ESP solutions, you just need to define the exported environment variables in your project's `.vscode/settings.json` file using the `idf.customExtraVars` VS Code configuration setting.

For example, for [ESP-Matter](#) you need to define the `ESP_MATTER_PATH` variable with the path to your local ESP-Matter repository:

```
{
  "idf.customExtraVars": {
    "ESP_MATTER_PATH": "/path/to/esp-matter"
  }
}
```

or you can do it from the Settings UI of Visual Studio Code:

Navigate to View > Command Palette.

Type Preferences: Open Settings (UI) and select the command to open the Settings UI. Select the workspace tab to edit the settings for the current workspace (your ESP-IDF project) or user tab to edit the settings for all VS Code instances.

Search for `idf custom extra vars` or `idf.customExtraVars`.

Click on Add Item and add the variable name (e.g., `ESP_MATTER_PATH`) and its value (e.g., `/path/to/esp-matter`).

C and C++ Code Navigation and Syntax Highlighting

Note

C and C++ code navigation is automatically configured if you create the project as described in [Configure Your Project](#). Those commands generate the `{PROJECT_DIRECTORY_PATH}/.vscode/c_cpp_properties.json` file.

For code navigation and C/C++ syntax highlighting, you can use [Microsoft C/C++ extension](#), [LLVM clangd extension](#), or any other preferred extension.

Usually C/C++ language extensions rely on a file called `compile_commands.json`, which is located in your project build directory. You can generate this file using the ESP-IDF: Run `idf.py reconfigure` task.

For [LLVM clangd extension](#), the user can use the ESP-IDF: Configure project for ESP-Clang to configure this extension argument. The command will search for `esp-clang` in the configured ESP-IDF setup, the build directory from `idf.buildPath` (`idf.buildPathWin` in Windows) and the GCC toolchain path from current `IDF_TARGET` and configured ESP-IDF setup and use these paths to configure clang path and arguments.

The result looks like this:

```
{
  "clangd.path": "/Users/user/.espressif/tools/esp-clang/esp-18.1.2_20240912/esp-clang/bin/clangd",
  "clangd.arguments": [
    "--background-index",
    "--query-driver=/Users/user/.espressif/tools/xtensa-esp-elf/esp-14.2.0_20241119/xtensa-esp-elf",
    "--compile-commands-dir=/path/to/esp-idf-project/build"
  ]
}
```

For [Microsoft C/C++ extension](#), a configuration file is located at `{PROJECT_DIRECTORY_PATH}/.vscode/c_cpp_properties.json`. This file can be generated by creating a project using ESP-IDF: New Project or by using the ESP-IDF: Add VS Code Configuration Folder command on an existing ESP-IDF project.

The file structure is as follows:

```
{
  "configurations": [
    {
      "name": "ESP-IDF",
      "compilerPath": "/path/to/toolchain-gcc",
      "compileCommands": "${workspaceFolder}/build/compile_commands.json",
      "includePath": [
        "/path/to/esp-idf/components/**",
        "${workspaceFolder}/**"
      ],
      "browse": {
        "path": [
          "/path/to/esp-idf/components",
          "${workspaceFolder}"
        ]
      }
    }
  ]
}
```

If `compile_commands.json` is not defined, Microsoft C/C++ extension will browse the provided ESP-IDF path to resolve code navigation.

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Build Your Project

[ESP-IDF Extension for VSCode](#)

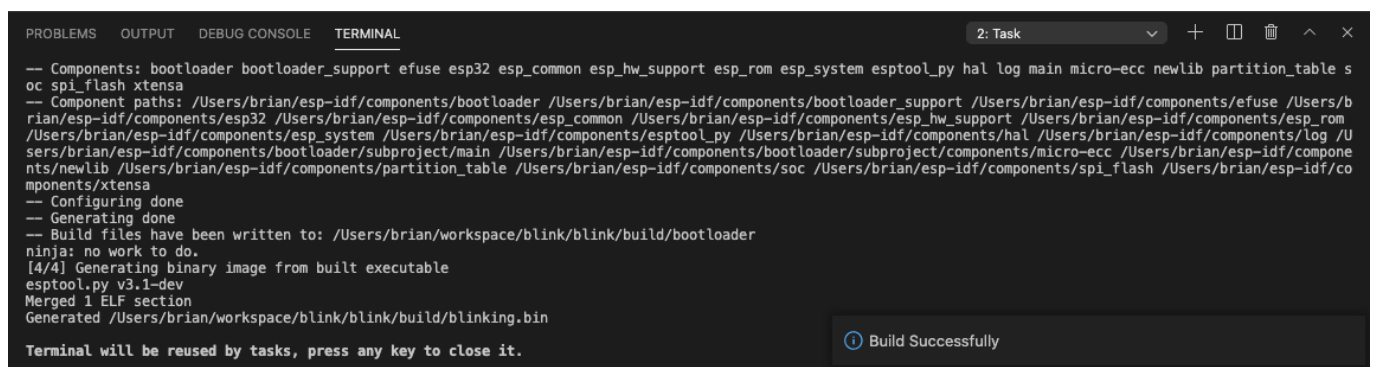
- »
- Build Your Project
- [Edit on GitHub](#)

Build Your Project

[\[ä.æ\]](#)

Build your project: Go to View > Command Palette, enter ESP-IDF: Build your Project and choose the command to build the project.

A new terminal will open with the build output. A notification bar will display the `Building project` message, followed by a `Building done` message when the process is complete.



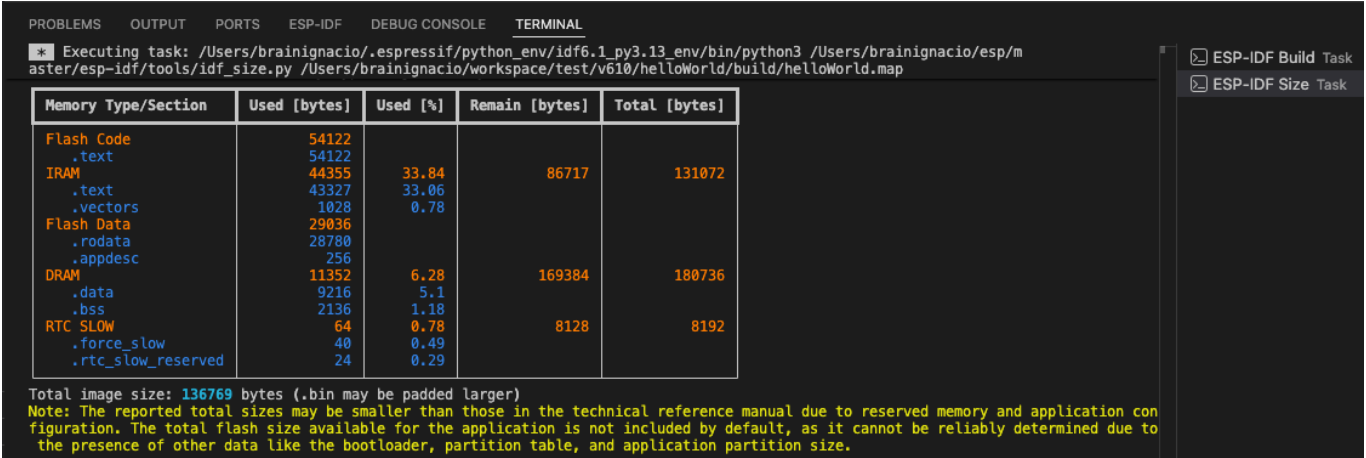
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL 2: Task
-- Components: bootloader bootloader_support efuse esp32 esp_common esp_hw_support esp_rom esp_system esptool_py hal log main micro-ecc newlib partition_table s
oc spi_flash xtensa
-- Component paths: /Users/brian/esp-idf/components/bootloader /Users/brian/esp-idf/components/bootloader_support /Users/brian/esp-idf/components/efuse /Users/b
rian/esp-idf/components/esp32 /Users/brian/esp-idf/components/esp_common /Users/brian/esp-idf/components/esp_hw_support /Users/brian/esp-idf/components/esp_rom
/Users/brian/esp-idf/components/esp_system /Users/brian/esp-idf/components/esptool_py /Users/brian/esp-idf/components/hal /Users/brian/esp-idf/components/log /U
sers/brian/esp-idf/components/bootloader/subproject/main /Users/brian/esp-idf/components/bootloader/subproject/components/micro-ecc /Users/brian/esp-idf/compone
nts/newlib /Users/brian/esp-idf/components/partition_table /Users/brian/esp-idf/components/soc /Users/brian/esp-idf/components/spi_flash /Users/brian/esp-idf/co
mponents/xtensa
-- Configuring done
-- Generating done
-- Build files have been written to: /Users/brian/workspace/blink/blink/build/bootloader
ninja: no work to do.
[4/4] Generating binary image from built executable
esptool.py v3.1-dev
Merged 1 ELF section
Generated /Users/brian/workspace/blink/blink/build/blink.bin
Terminal will be reused by tasks, press any key to close it.
Build Successfully
```

Note

There is an `idf.notificationMode` configuration setting if you prefer not to see the output automatically. Please refer to [ESP-IDF Settings](#) to learn how to modify this configuration.

You can adjust the behavior of the build task with `idf.cmakeCompilerArgs` for the Cmake configure step and `idf.ninjaArgs` for the Ninja step. For example, use `idf.ninjaArgs: [-j N]` where **N** is the number of jobs to run in parallel.

After building the application, the size analysis task will run in a terminal, showing the binary size analysis results. You can enable or disable this task with `idf.enableSizeTaskAfterBuildTask` in the `settings.json`.

3. 

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	54122			
.text	54122			
IRAM	44355	33.84	86717	131072
.text	43327	33.06		
.vectors	1028	0.78		
Flash Data	29036			
.rodata	28788			
.appdesc	256			
DRAM	11352	6.28	169384	180736
.data	9216	5.1		
.bss	2136	1.18		
RTC SLOW	64	0.78	8128	8192
.force_slow	40	0.49		
.rtc_slow_reserved	24	0.29		

Total image size: 136769 bytes (.bin may be padded larger)
Note: The reported total sizes may be smaller than those in the technical reference manual due to reserved memory and application configuration. The total flash size available for the application is not included by default, as it cannot be reliably determined due to the presence of other data like the bootloader, partition table, and application partition size.

The ESP-IDF build output will provide hints to resolve errors, which can be viewed using the [Hints Viewer](#).

Next, proceed to [Flash onto the Device](#).

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Flash onto the Device

[ESP-IDF Extension for VSCode](#)

- »
- Flash onto the Device
- [Edit on GitHub](#)

Flash onto the Device

[æ]

Select the serial port: Go to View > Command Palette, enter ESP-IDF: Select Port to Use, and choose the command to specify the serial port of your device.

Start flashing the device: Go to View > Command Palette, enter ESP-IDF: Flash your Project, and choose the command to start flashing your device. You can choose UART, JTAG or DFU flashing method.

Note

UART is the most common option for most Espressif devices.

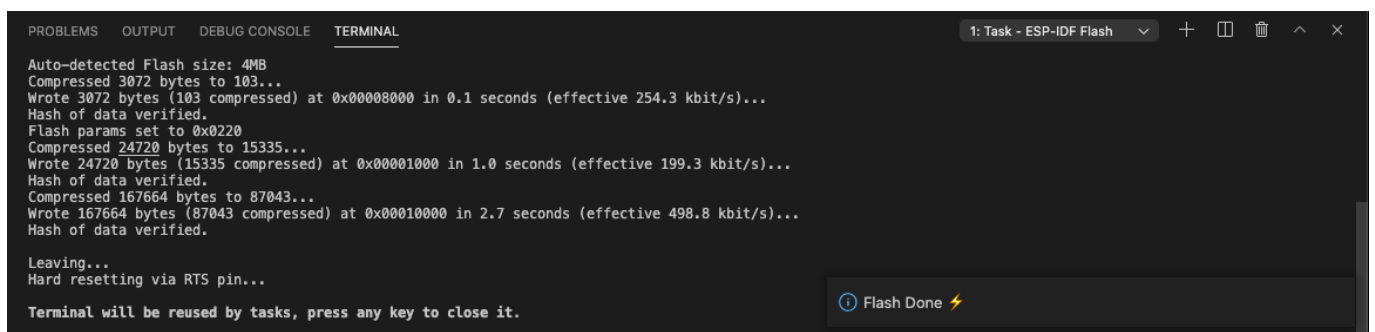
If you are using JTAG, ensure OpenOCD is properly configured. Go to View > Command Palette and enter ESP-IDF: Select OpenOCD Board Configuration to select the correct configuration for your board.

Note

There is an `idf.flashBaudRate` configuration setting to modify the flashing baud rate.

The flashing task will begin on the selected serial port, launching a new terminal displaying the flashing task output. While flashing is in progress, a notification bar will display ESP-IDF: Flashing project. Once the process is complete, the message will change to Flash Done.

3.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL 1: Task - ESP-IDF Flash
Auto-detected Flash size: 4MB
Compressed 3072 bytes to 103...
Wrote 3072 bytes (103 compressed) at 0x00000000 in 0.1 seconds (effective 254.3 kbit/s)...
Hash of data verified.
Flash params set to 0x0020
Compressed 24720 bytes to 15335...
Wrote 24720 bytes (15335 compressed) at 0x00001000 in 1.0 seconds (effective 199.3 kbit/s)...
Hash of data verified.
Compressed 167664 bytes to 87043...
Wrote 167664 bytes (87043 compressed) at 0x00010000 in 2.7 seconds (effective 498.8 kbit/s)...
Hash of data verified.
Leaving...
Hard resetting via RTS pin...
Terminal will be reused by tasks, press any key to close it.
Flash Done ⚡
```

Please refer to [Configuration of OpenOCD for Specific Target](#) to select the appropriate OpenOCD configuration file based on your hardware.

Make sure to configure your drivers as mentioned in [Configure JTAG Interface](#).

Next, proceed to [Monitor the Output](#).

Other Flashing Commands

You can also choose the flashing method UART, JTAG, or DFU (ESP32-S2 or ESP32-S3) based on your hardware connection.

Go to View > Command Palette, enter ESP-IDF: Select Flash Method, and select the flashing method for your device. The selected choice will be saved in the `idf.flashType` configuration setting.

You can also use the following commands to flash with a specific flashing method:

ESP-IDF: Flash (UART) your Project for UART flashing

ESP-IDF: Flash (DFU) your Project to flash using DFU (only for ESP32-S2 and ESP32-S3)

ESP-IDF: Flash (with JTAG) to flash using JTAG and OpenOCD

Related Documentation

[ESP-PROG Guide](#) for hardware connection and configuration

[Configuration of OpenOCD for Specific Target](#) to select the appropriate OpenOCD configuration file based on your hardware

[Establishing Serial Connection](#) with Espressif serial device

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Monitor the Output

[ESP-IDF Extension for VSCode](#)

- »
 - Monitor the Output
 - [Edit on GitHub](#)
-

Monitor the Output

[ä,æ]

To view the serial monitor output from your device, please follow the instructions below:

Select the serial port:

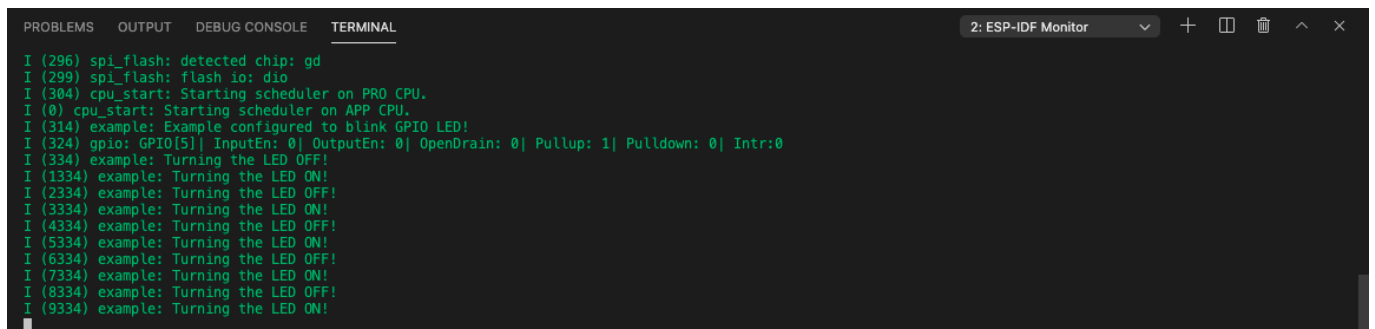
Go to View > Command Palette

Enter ESP-IDF: Select Port to Use and choose the command to specify the serial port of your device

Start monitoring:

Go to View > Command Palette

Enter ESP-IDF: Monitor Device and choose the command to start monitoring your device

A screenshot of the ESP-IDF Monitor terminal window. The window has a title bar with '2: ESP-IDF Monitor' and standard window controls. The terminal shows a series of log messages: 'I (296) spi_flash: detected chip: gd', 'I (299) spi_flash: flash io: dio', 'I (304) cpu_start: Starting scheduler on PRO CPU.', 'I (0) cpu_start: Starting scheduler on APP CPU.', 'I (314) example: Example configured to blink GPIO LED!', 'I (324) gpio: GPIO[5] InputEn: 0 OutputEn: 0 OpenDrain: 0 Pullup: 1 Pulldown: 0 Intr:0', 'I (334) example: Turning the LED OFF!', 'I (1334) example: Turning the LED ON!', 'I (2334) example: Turning the LED OFF!', 'I (3334) example: Turning the LED ON!', 'I (4334) example: Turning the LED OFF!', 'I (5334) example: Turning the LED ON!', 'I (6334) example: Turning the LED OFF!', 'I (7334) example: Turning the LED ON!', 'I (8334) example: Turning the LED OFF!', and 'I (9334) example: Turning the LED ON!'. The messages are timestamped and show the state of an LED being toggled every 1000ms.

Note

The monitor baud rate is defined with `CONFIG_ESPTOOLPY_MONITOR_BAUD` in the project's SDK Configuration Editor. You can override it by setting a value in `idf.monitorBaudRate`.

Next, proceed to [debug your project](#).

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Debug Your Project

[ESP-IDF Extension for VSCode](#)

- »
 - Debug Your Project
 - [Edit on GitHub](#)
-

Debug Your Project

[ä.æ]

Table of Contents

Open an ESP-IDF project

First open an ESP-IDF project in Visual Studio Code. You can create a new ESP-IDF project by following the instructions in [Create an ESP-IDF Project](#) or open an existing ESP-IDF project.

Note

Please ensure the ESP-IDF project includes a Visual Studio Code debug configuration file (launch.json) in the <project-dir>/.vscode/launch.json file. If not, you can add the default configuration with the ESP-IDF: Add VS Code Configuration Folder command (Press F1 and search for the command).

Starting a Debug Session

Before debugging the project, you need to specify the serial port of the device:

Select the serial port

Go to View > Command Palette

Enter ESP-IDF: Select Port to Use and choose the command to specify the serial port of your device

Make sure that OpenOCD configuration files are correct

Go to View > Command Palette

Enter ESP-IDF: Select OpenOCD Board Configuration and choose the command to specify the OpenOCD configuration files for the extension OpenOCD server

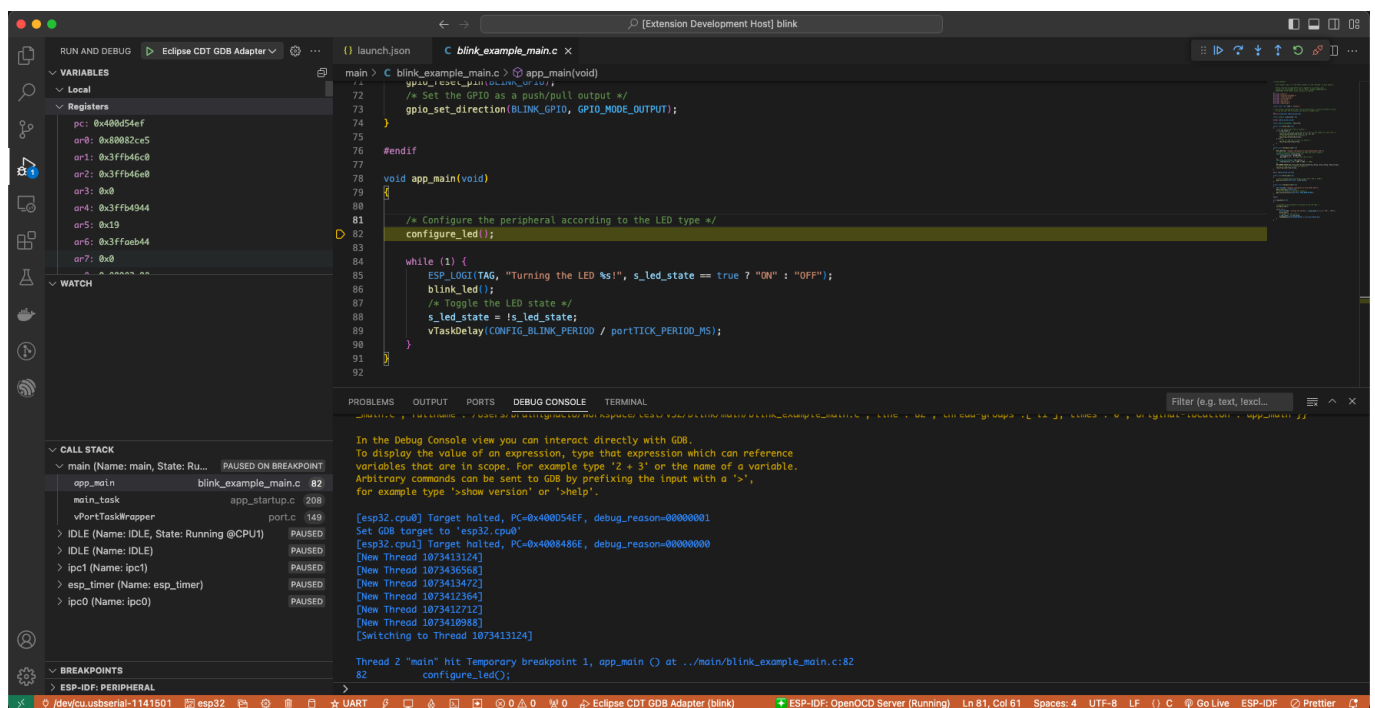
Note

Please review [Configuration of OpenOCD for Specific Target](#) to understand which board or configuration to use for your specific hardware.

Make sure to configure your drivers as mentioned in ESP-IDF [Configure JTAG Interface](#) documentation.

(Linux users) Copy the [OpenOCD udev rules files](#) into the `/etc/udev/rules.d` directory before running OpenOCD and starting a debug session.

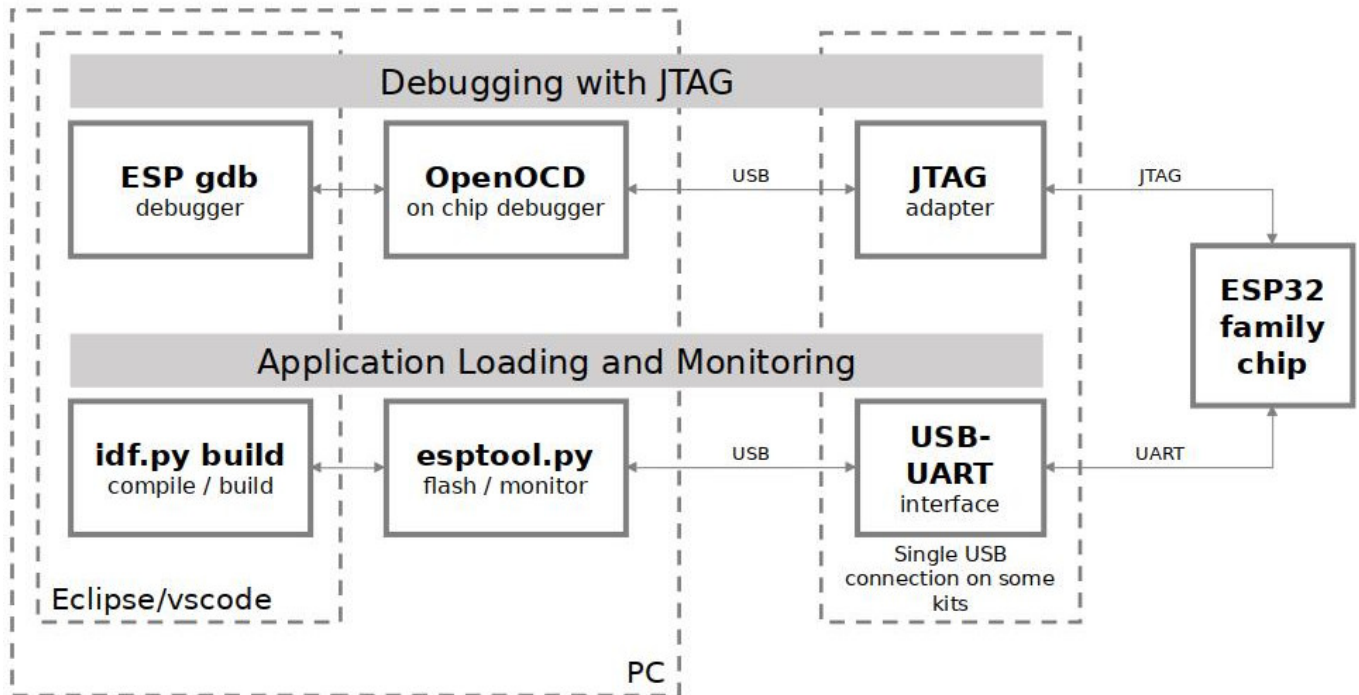
To start debugging, select Run > Start Debugging from the menu or press F5.



You can see the GDB output in the debug console and the OpenOCD output under view > output by selecting ESP-IDF from the dropdown menu.

The above covers the basic functionality of the ESP-IDF extension. For more information, please refer to [Additional IDE Features](#).

Debugging Process Overview



JTAG debugging overview diagram

First, the OpenOCD server is launched in the background and the output appears under menu view > output by selecting ESP-IDF from the dropdown menu.

By default, the OpenOCD server is launched on localhost, port 4444 for Telnet communication, port 6666 for TCL communication and port 3333 for GDB. You can change these settings by modifying `openocd.tcl.host` and `openocd.tcl.port` in your `<project-directory>/.vscode/settings.json`. You can also adjust the verbosity of OpenOCD messages displayed in the ESP-IDF output by setting `idf.openOcdDebugLevel`. Values from 0 (error messages only) to 4 (Verbose low-level debug message).

Next, the [Eclipse CDT GDB Adapter](#) is launched in the background, with its output shown in the Debug Console. This adapter initiates the GDB debug session to connect to the device.

This adapter acts as an intermediary between Visual Studio Code, the configured toolchain GDB and the OpenOCD server. Learn more about [Espressif chips debugging](#) and how Visual Studio Code uses [debug adapters](#) to communicate with various debug tools.

Note

Use `idf.openOcdDebugLevel` configuration setting to 4 or more to show debug logging in OpenOCD server output.

Use `verbose` in your `<project-directory>/.vscode/launch.json` to true to show more debug adapter output.

By default OpenOCD arguments are `openocd -d${idf.openOcdDebugLevel} -f ${idf.openOcdConfigs}`. If you want to modify them, set `idf.openOcdLaunchArgs`, an empty array by default, to override the default arguments.

For example, to add `-c "init"` to the OpenOCD command, set `idf.openOcdLaunchArgs` in your `<project-directory>/vscode/settings.json` as follows:

```
{
  "idf.openOcdLaunchArgs": [
    "-d${config:idf.openOcdDebugLevel}",
    "${config:idf.openOcdConfigs, -f}"
    "-c",
    "init",
    "-c",
    "reset halt",
  ]
}
```

In Visual Studio Code select menu **View > Output > ESP-IDF**. This output information is useful to know what is happening in the extension. For example, openocd communication is displayed in this output.

Using a Custom OpenOCD Path

By default the extension will use the OpenOCD installed in the ESP-IDF `IDF_TOOLS_PATH` directory. If you want to use a custom OpenOCD, you can set `idf.customOpenOCDPath` in your `<project-directory>/vscode/settings.json`. If this setting is empty or points to a path that does not exist, the extension will fallback to searching OpenOCD in `PATH`.

```
{
  "idf.customOpenOCDPath": "/path/to/your/openocd"
}
```

Setting a Custom Application Image Offset

If you modify the application image offset, you need to update OpenOCD launch arguments accordingly. This is necessary if the OpenOCD output (Menu `View > Output > ESP-IDF`) shows an error like:

```
Failed to get flash maps (-6)!
â■■ Error: Failed to get flash maps (-6)!
Warn : Application image is invalid! Check configured binary flash offset 'appimage_offset'.
```

To update OpenOCD launch arguments, open the project's `vscode/settings.json` and add or modify:

```
{
  "idf.openOcdLaunchArgs": [
    "-d${config:idf.openOcdDebugLevel}",
    "${config:idf.openOcdConfigs, -f}"
    "-c",
    "init",
    "-c",
    "reset halt",
  ]
}
```

```

        "-c",
        "esp appimage_offset 0x20000"
    ]
}

```

where 0x20000 is your application image offset used in the partition table.

Debugging Configuration

To configure the debugging session, open the project's `.vscode/launch.json` file. This file contains the configuration for the debug session. The default configuration is as follows:

```

{
  "configurations": [
    {
      "type": "gdbtarget",
      "request": "attach",
      "name": "Eclipse CDT GDB Adapter"
    }
  ]
}

```

You can modify the configuration to suit your needs. Let's describe the configuration options:

type: The type of the debug configuration. It should be set to `gdbtarget`.

program: ELF file of your project build directory to execute the debug session. You can use the command `${command:espIdf.getProjectName}` to query the extension to find the current build directory project name.

initCommands: Optional GDB commands run **after** the target is connected via `target.connectCommands`. The extension does not inject a default `initCommands` list.

initialBreakpoint: When set to a non-empty string (for example `app_main`), the extension appends `thb <value>` to `initCommands`. When set to `""` (empty string), the session does not stop on an initial breakpoint. When omitted, the extension adds `thbreak app_main` to the connect sequence.

gdb: GDB executable to be used. By default `â€œ${command:espIdf.getToolchainGdb}â€œ` will query the extension to find the ESP-IDF toolchain GDB for the current `IDF_TARGET` of your `esp-idf` project (`esp32`, `esp32c6`, etc.).

Note

GDB command order. The Eclipse CDT GDB Adapter runs `target.connectCommands` first to attach to the target, then runs `initCommands`.

The extension **prepends** commands to `target.connectCommands` (before any user-defined `connectCommands`), in this order:

A source line for each ESP-IDF generated gdbinit file listed in `gdbinit_files` of `build/project_description.json`, using the same order as `idf.py gdb: symbols, prefix_map` and `py_extensions`. `py_extensions` is skipped when the configured GDB has no Python support. When `gdbinit_files` is unavailable, the extension looks for those files under `${idf.buildPath}/gdbinit` and falls back to `prefix_map_gdbinit` for path remapping.

The commands from ESP-IDF's `gdbinit/connect` file, replayed one by one instead of being sourced, with these adjustments:

The trailing `continue` is dropped. Connect commands run before the debug session is initialized, so resuming the target here would run past breakpoints the editor has not inserted yet and the resulting stop would not be reported.

`thbreak app_main` is dropped when `initialBreakpoint` is defined in the launch configuration, so that setting alone decides where the session first stops.

`target remote :3333` is rewritten when `target.host` or `target.port` are defined.

When the file is missing or has a shape the extension cannot expand, such as the `CONFIG_IDF_TARGET_LINUX` variant that has no remote target, the extension falls back to an equivalent sequence:

```
set remotetimeout 10
```

```
target remote :3333 â uses target.host and target.port when they are defined
```

```
monitor reset halt
```

```
maintenance flush register-cache
```

```
thbreak app_main â only when initialBreakpoint is omitted from the launch configuration
```

Core dump and GDB Stub post-mortem sessions get only the source lines, since they provide their own `connectCommands`. If `CONFIG_APP_REPRODUCIBLE_BUILD` is enabled and no prefix map file is found, the extension shows an information message.

Some additional arguments you might use are:

`buildFlashMonitor`: (Default: false). Build, flash and launch IDF Monitor before starting the debug session. Use `idf.monitorDelay` to set a delay, in milliseconds, after starting the monitor (Default is 1000).

`runOpenOCD`: (Default: true). Run extension OpenOCD Server.

`verifyAppBinBeforeDebug`: (Default: false) Verify that current ESP-IDF project binary is the same as binary in chip.

`logFile`: Absolute path to the file to log interaction with gdb. Example: `${workspaceFolder}/gdb.log`.

`verbose`: Produce verbose log output.

environment: Environment variables to apply to the ESP-IDF Debug Adapter. It will replace global environment variables and environment variables used by the extension.

```
{
  "environment": {
    "VAR": "Value"
  }
}
```

imageAndSymbols :

```
{
  "imageAndSymbols": {
    "symbolFileName": "If specified, a symbol file to load at the given (optional) offset",
    "symbolOffset": "If symbolFileName is specified, the offset used to load",
    "imageFileName": "If specified, an image file to load at the given (optional) offset",
    "imageOffset": "If imageFileName is specified, the offset used to load"
  }
}
```

target: Configuration for target to be attached. Specifies how to connect to the device to debug. Usually OpenOCD exposes the chip as a remote target on port 3333.

```
{
  "target": {
    "type": "The kind of target debugging to do. This is passed to -target-select (defaults to remote)",
    "host": "Target host to connect to (defaults to 'localhost', ignored if parameters is set)",
    "port": "Target port to connect to (defaults to value captured by serverPortRegExp, ignored if parameters is set)",
    "parameters": "Target parameters for the type of target. Normally something like localhost:1234",
    "connectCommands": "Array of GDB commands to establish the connection. The gdbinit ``source``"
  }
}
```

An example of a customized launch.json is shown below. The minimal default configuration above is enough for most projects, since the extension resolves the ESP-IDF generated gdbinit files and the connect sequence on its own. Optional fields can be added as needed:

```
{
  "configurations": [
    {
      "type": "gdbtarget",
      "request": "attach",
      "name": "Eclipse CDT GDB Adapter",
      "program": "${workspaceFolder}/build/${command:espIdf.getProjectName}.elf",
      "gdb": "${command:espIdf.getToolchainGdb}",
      "initialBreakpoint": "app_main",
      "initCommands": [
        "set remote hardware-watchpoint-limit 2"
      ],
    }
  ]
}
```

```

    "target": {
      "connectCommands": [
        "set remotetimeout 20"
      ]
    }
  ]
}

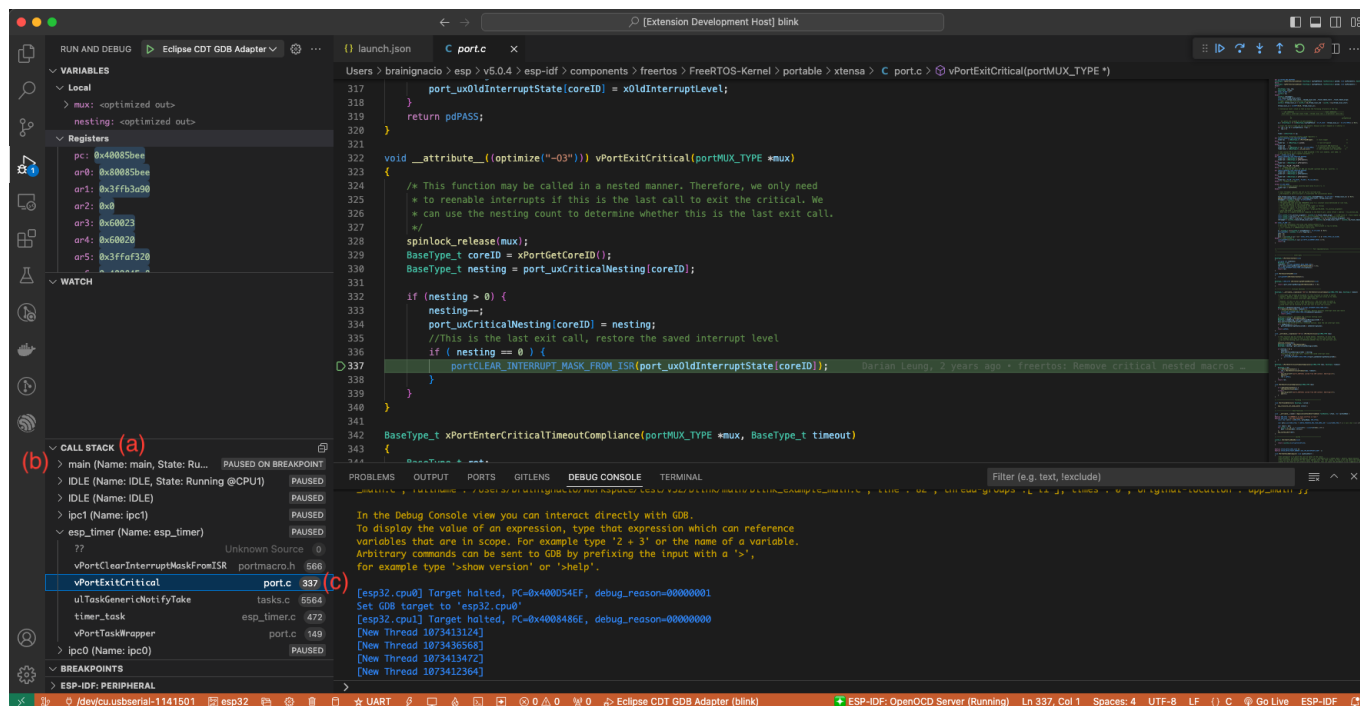
```

There are other, less used arguments documented in the ESP-IDF VS Code extension's `package.json` `gdbtarget` debugger contribution.

Navigating through the Code, Call Stack and Threads

When the target halts, the editor will show the line of code where the program halts and the list of threads in the `CALL STACK` sub-window (a) on the `Run` icon in the Activity Bar on the side of Visual Studio Code. The first line of call stack under `main` (b) contains the last called function `app_main()`, which in turn was called from `main_task()` as shown in the previous image. Each line of the stack also contains the file name and line number (c) where the function was called. By clicking on each of the stack entries, you will see the file opened.

By expanding threads, you can navigate throughout the application. Some threads contain longer call stacks where you can see, besides function calls, numbers like `0x4000bff0`, representing addresses of binary code not provided in source form.

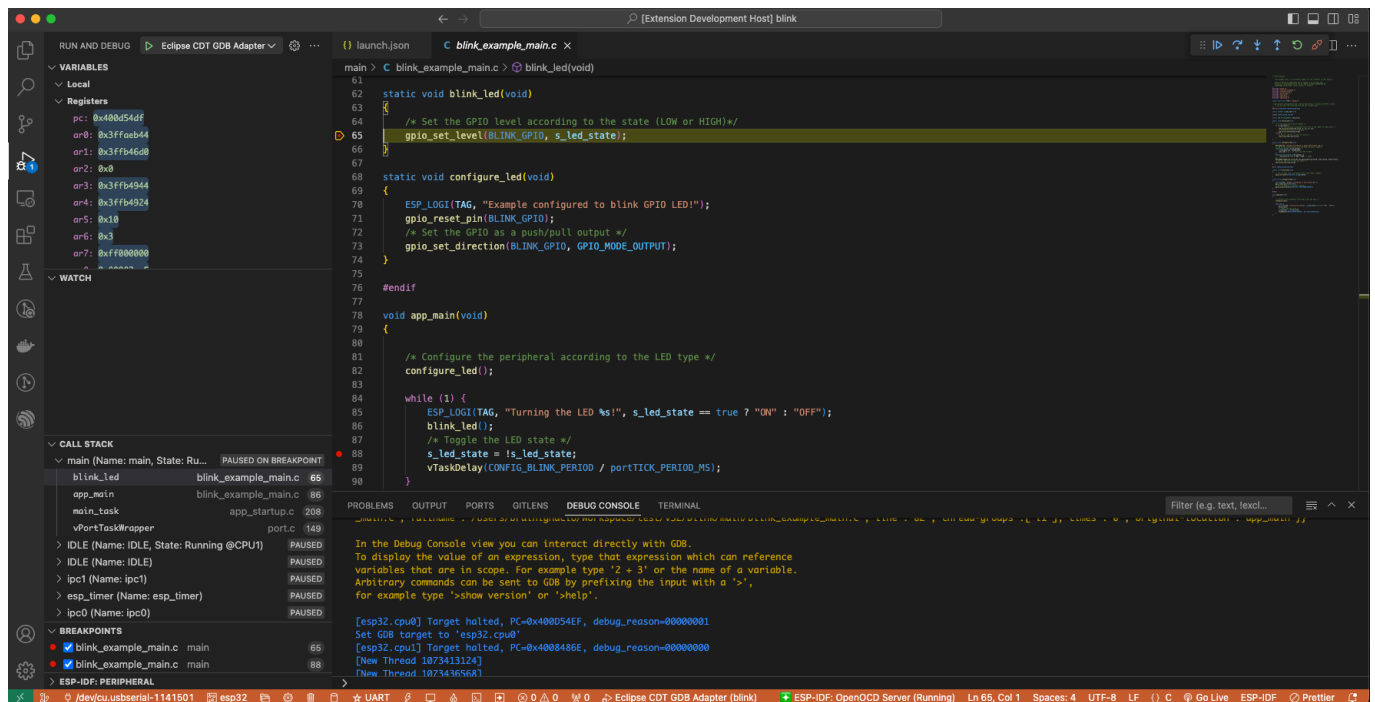


Go back to the `app_main()` in Thread #1 to familiarize yourself with the code in the `blink.c` file, which will be examined in more detail in the following examples. Debugger makes it easy to navigate through the code of entire application. This is useful when stepping through the code and working with breakpoints, as will be discussed below.

Setting and Clearing Breakpoints

When debugging, you often need to pause the application at critical points in the code to examine the state of specific variables, memory, registers and peripherals. To achieve this, you can use breakpoints, which provide a convenient way to quickly halt the application at a specific line of code.

For example, establish two breakpoints where the state of LED changes. Based on the code listing below, this happens at lines 57 and 80. To set a breakpoint, go to the desired line and press F9 or click on the circle shown next to the line number in the editor margin. The list of breakpoints appears in the Breakpoints sub-window under the Run icon in the Activity Bar on the side of Visual Studio Code.



Note

ESP32 supports a maximum of two hardware breakpoints. For more information, refer to [Breakpoints and Watchpoints Available](#).

Once a debug session starts, a **debug toolbar** will appear on the top of the VS Code editor with several actions, as explained in [Visual Studio Code Debug Actions](#).

Press F5 (Continue/Pause), the processor will run and halt at the next breakpoint. Press F5 again to stop at the next breakpoint, and so on. You can observe that the LED changes the state after each **Continue** command.

Learn more about breakpoints under [What Else Should I Know About Breakpoints?](#).

Halting the Target Manually

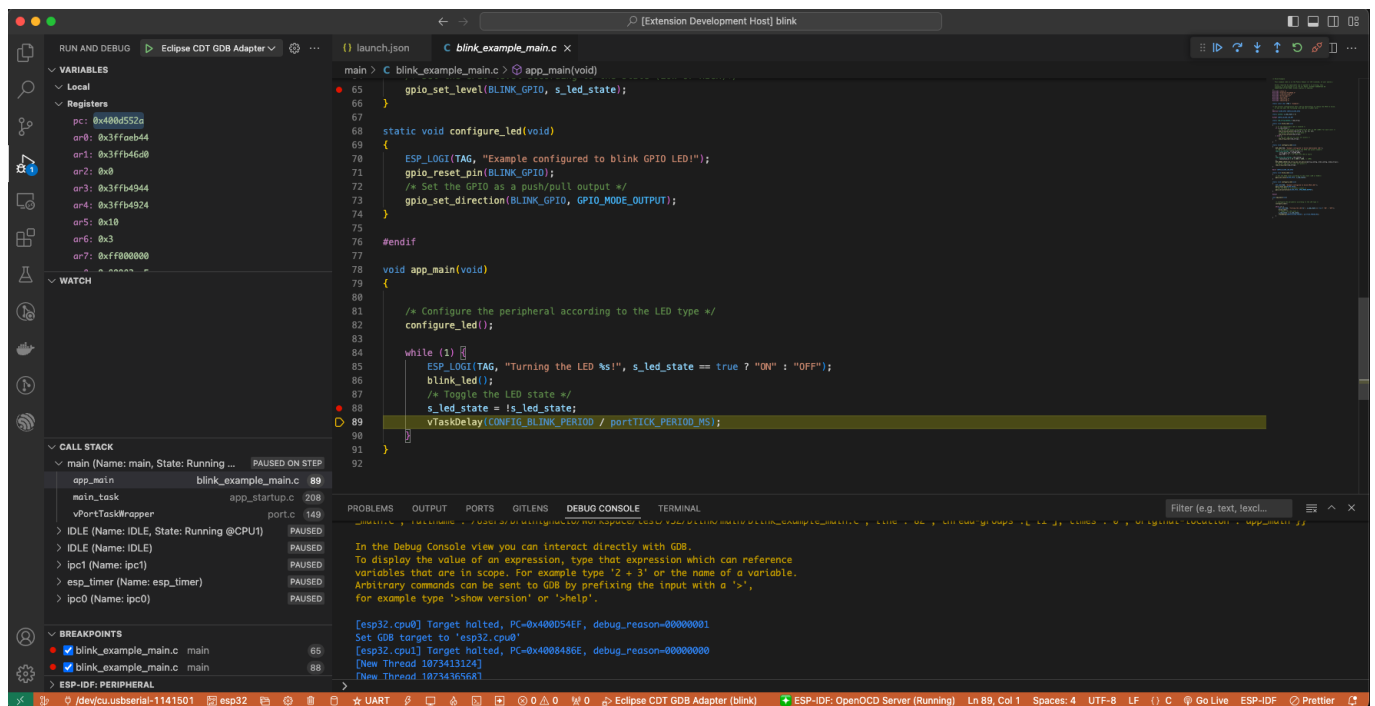
When debugging, you may resume the application and enter code that waits for some event or stays in infinite loop without any break points defined. In such cases, to go back to debugging mode, you can break program execution manually by pressing **Continue/Pause** button. To check it, delete all

breakpoints and click **Continue**. Then click **Pause**. Application will halt at some random point and the LED will stop blinking.

You can also step through the code using the **Step Into (F11)** and **Step Over (F10)** commands. The difference is that **Step Into (F11)** enters inside subroutine calls, while **Step Over (F10)** treats it as a single source line.

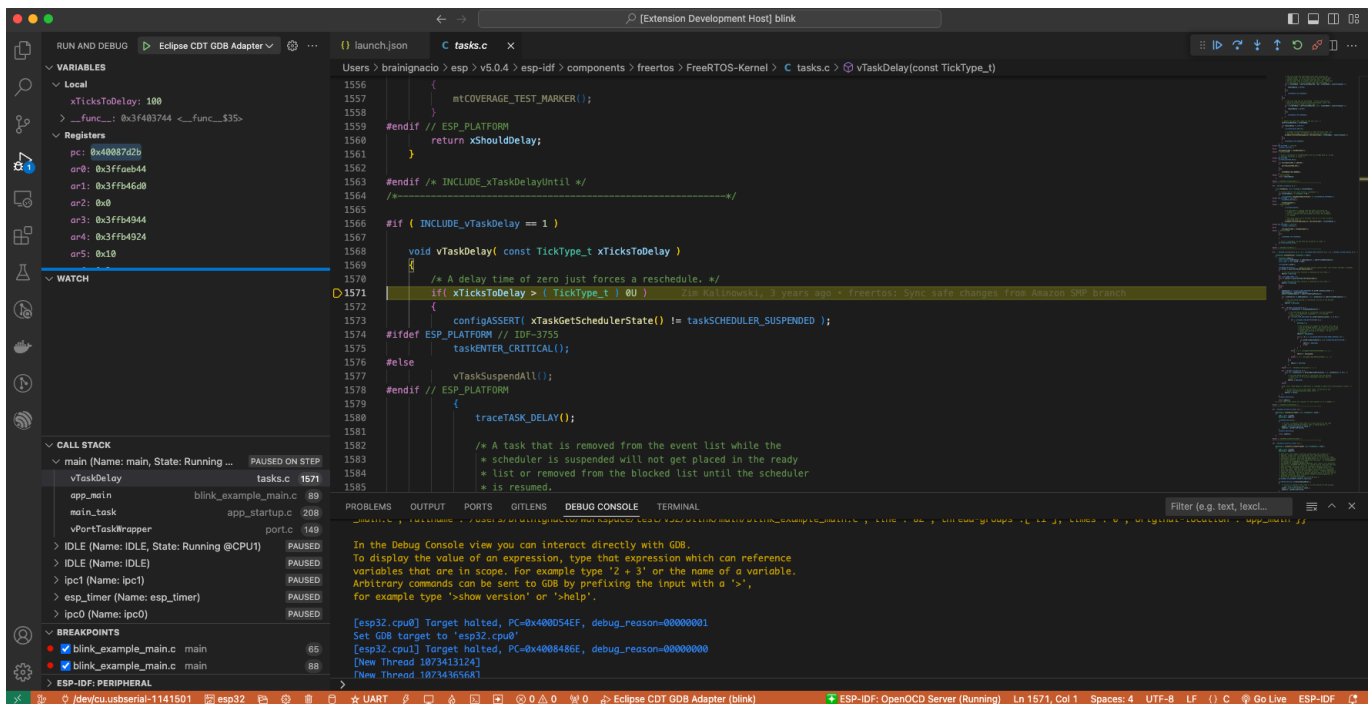
Before demonstrating this functionality, make sure that you have only one breakpoint defined at line 57 of `blink.c` using information discussed in previous paragraphs.

Resume the program by pressing F5 and let it halt. Now press **Step Over (F10)** a few times to see how the debugger steps through the program one line at a time.



Stepping Through the Code

If you press **Step Into (F11)** instead, then debugger will step inside the subroutine call.

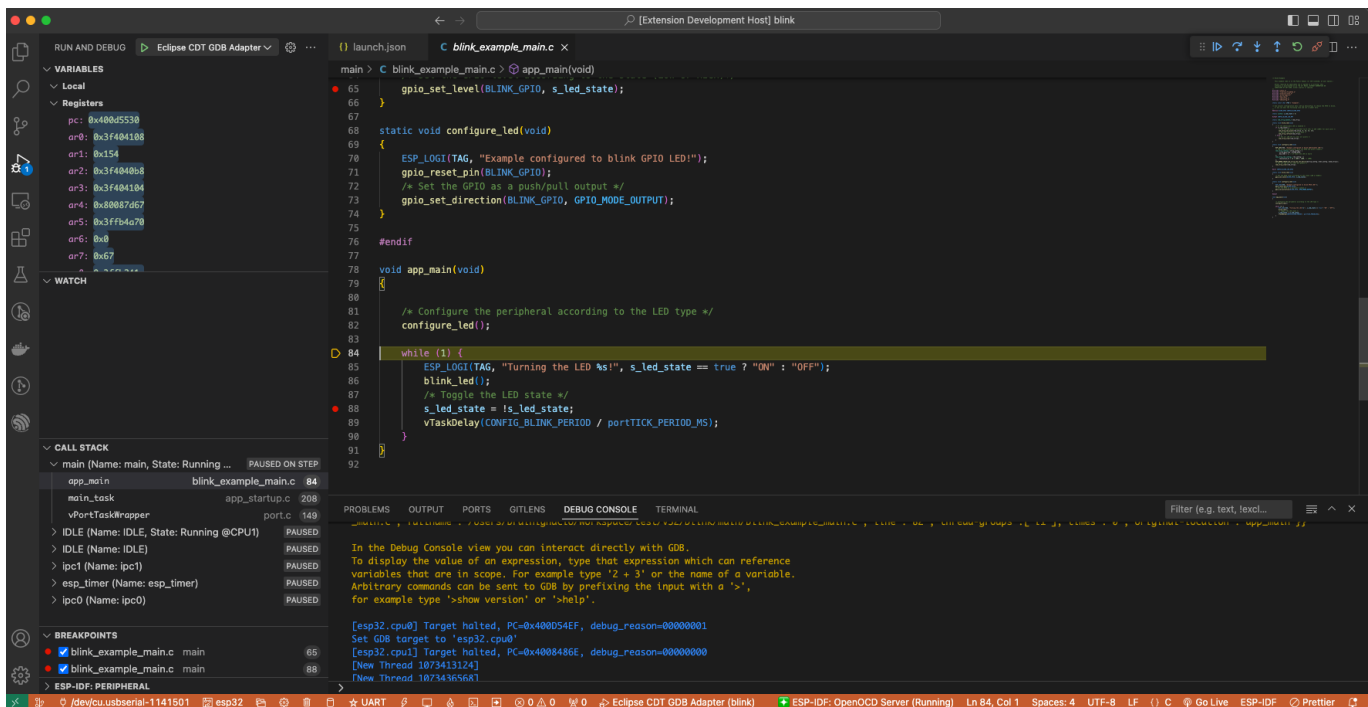


In this case, the debugger steps inside `vTaskDelay(CONFIG_BLINK_PERIOD / portTICK_PERIOD_MS)` and effectively moves to the `tasks.c` code.

Note

See [Why Stepping with `ânextâ` Does Not Bypass Subroutine Calls?](#) for potential limitations using the next command.

If you press `âStep Out (Shift + F11)â` instead, then debugger will step outside the subroutine call.



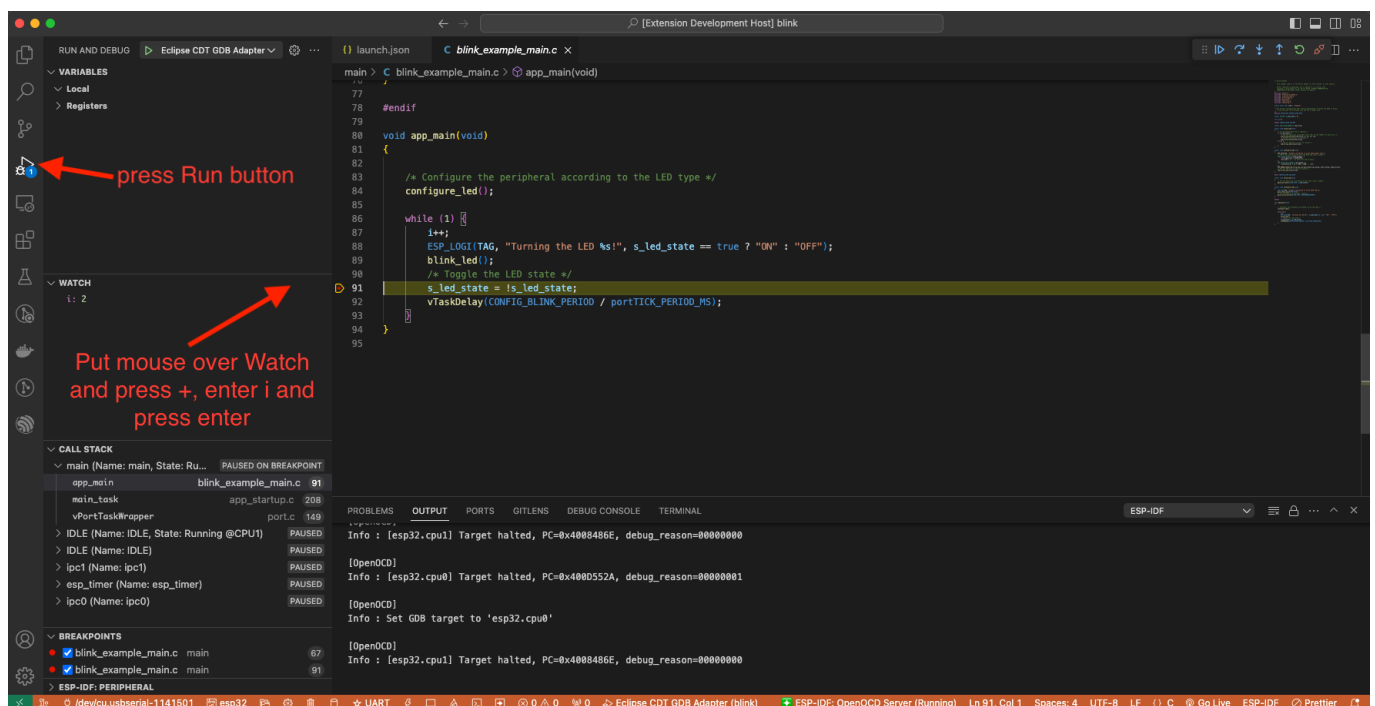
Watching and Setting Program Variables

A common debugging task is checking the value of a program variable as the program runs. To demonstrate this functionality, update file `blink.c` by declaring a global variable `int i` above the definition of the function `blink_task`. Then add `i++` inside `while(1)` of this function to increment `i` on each blink.

Stop debugging by pressing **Stop (Shift + F5)**. Build and flash the code to the target chip, then restart the debugger by pressing **F5**. Once the application halts, set a breakpoint on the line where `i++` is located.

In the watch sub-window on the Run icon in the Activity Bar on the side of Visual Studio Code, click the **+** and enter `i` to start watching its value.

Resume program execution by pressing **F5**. Each time the program pauses, the value of `i` will have incremented.

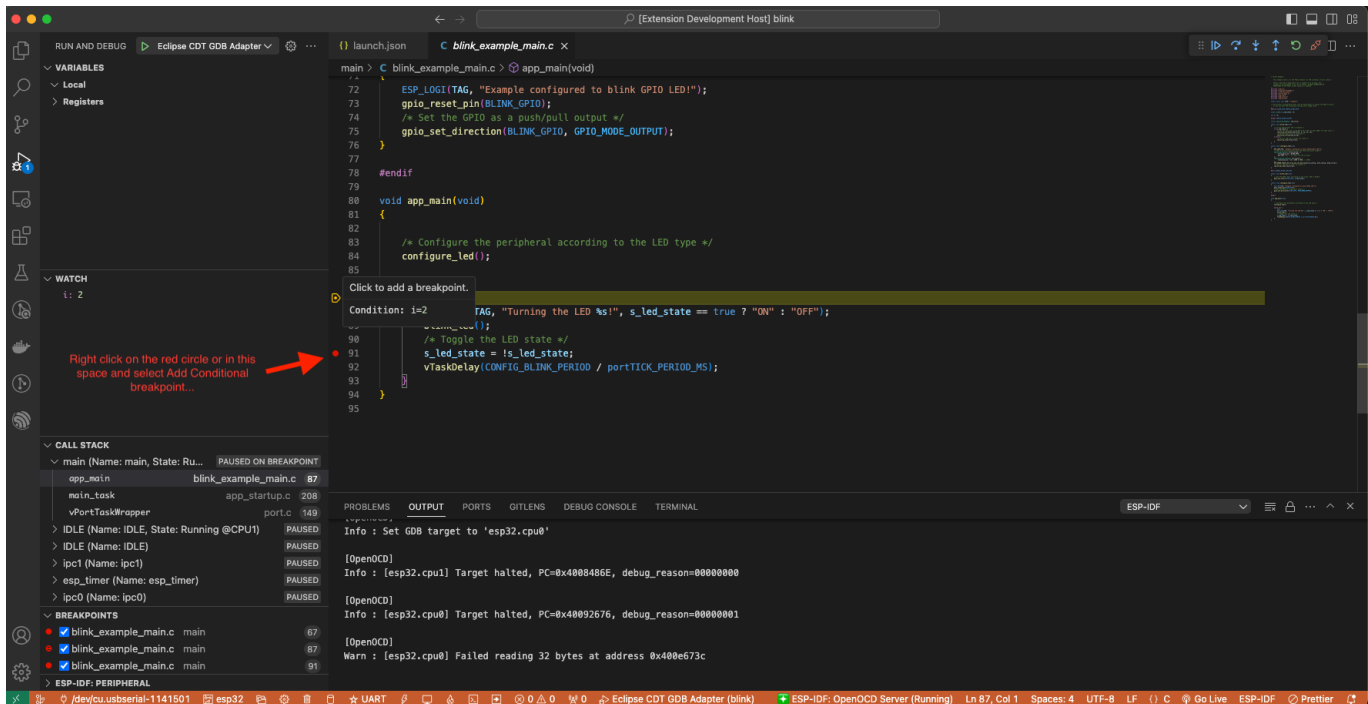


Setting Conditional Breakpoints

You can also set a breakpoint to halt the program execution if a certain condition is satisfied. See [Visual Studio Code Conditional Breakpoints](#).

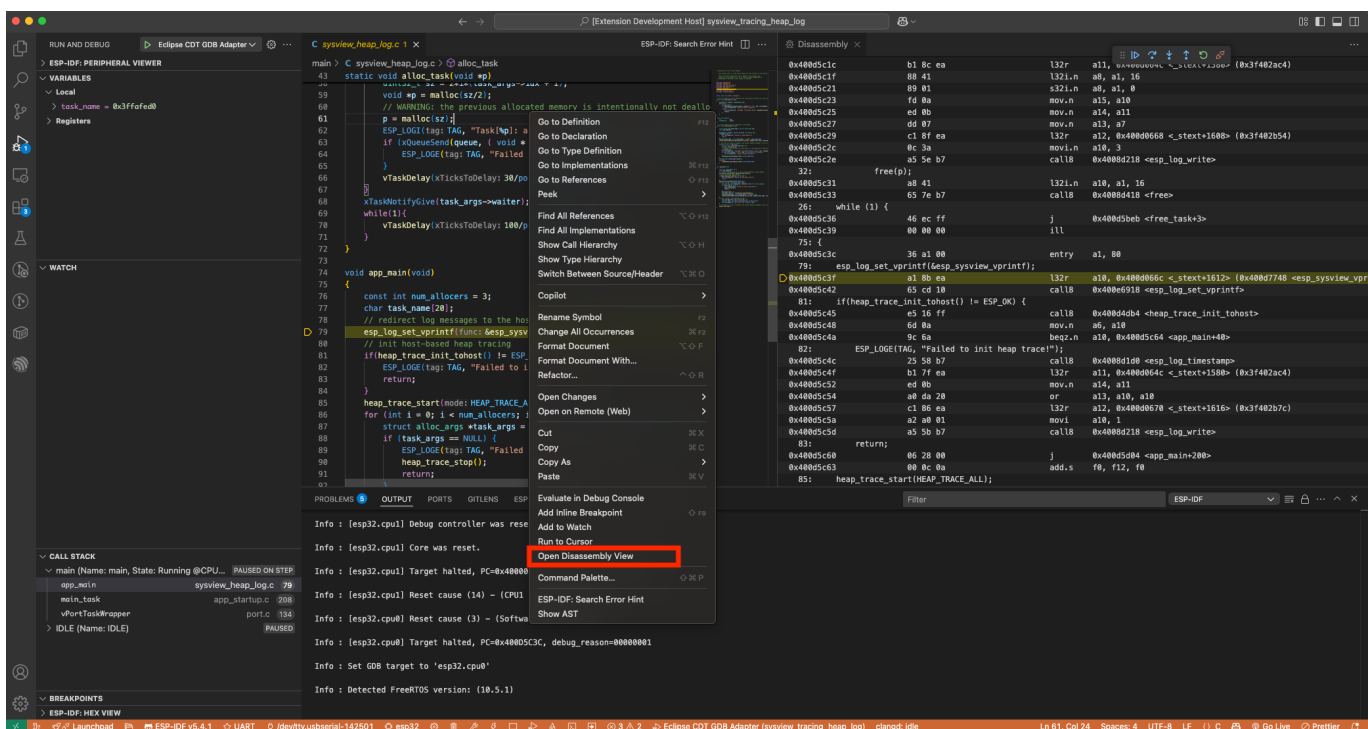
To set a new conditional breakpoint, go to the desired line, right-click on the circle next to the line number (editor margin), and select **Add Conditional Breakpoint** action. You can also modify a breakpoint to add a condition in the list of breakpoints in the Breakpoints sub-window on the Run icon in the Activity Bar. Click the pencil icon on the breakpoint and set the breakpoint condition.

For this example, go to line 79, right-click on the circle next to the line number (editor margin), select **Add Conditional Breakpoint** action, and set `i=2`. When you start debugging, the debugger will stop on line 79 when `i` equals 2.



Disassembly View

You can check the assembly code during a debugging session by right-clicking on any line in a source code file and selecting open Disassembly view. **Disassemble View** shows the assembly code with C code, where you can also set breakpoints.



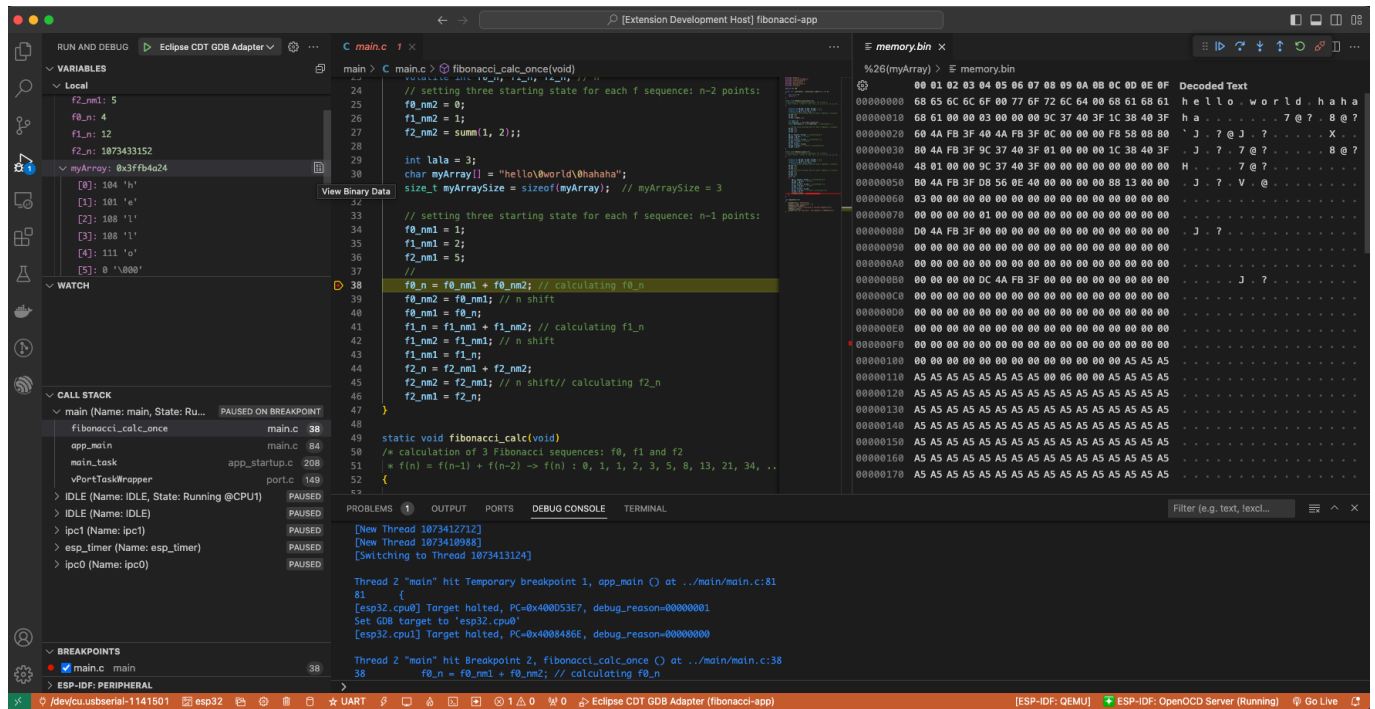
Watchpoints (Data Breakpoints)

See [Breakpoints and Watchpoints Available](#) for more information.

Send Commands to GDB

You can send any GDB command in the debug console with `> COMMAND`. For example, `> i threads`.

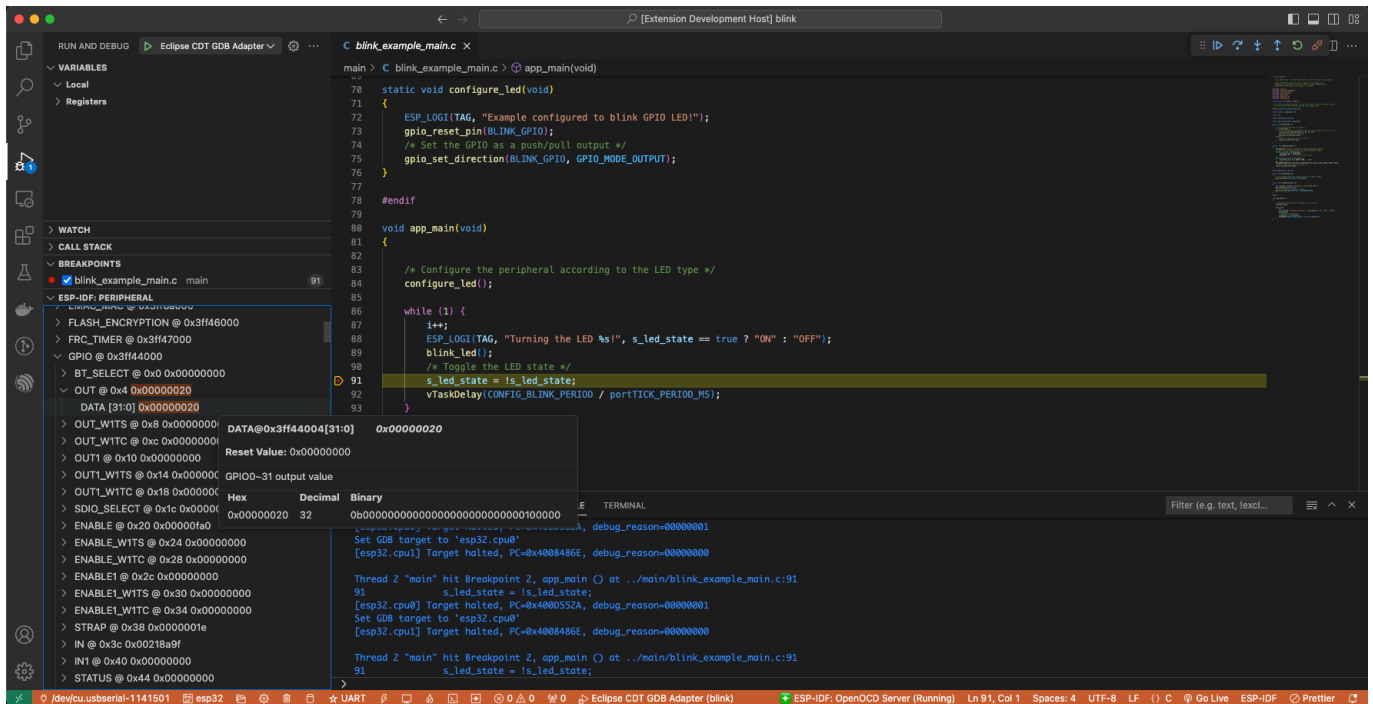
To view binary data variables, click view Binary Data next to the variable name.



Learn more about [Command Line Debugging](#).

ESP-IDF: Peripheral View

ESP-IDF extension provides an ESP-IDF: Peripheral View tree in the Run and Debug view. This tree view uses the SVD file specified in the IDF SVD File Path (`idf.svdFilePath`) configuration to populate a set of peripheral register values for the active debug session target. You can download Espressif SVD files from [Espressif SVD](#) repository.



Post-mortem Debugging Use Cases

You can start a monitor session to capture fatal error events with **ESP-IDF: Launch IDF Monitor for Core Dump Mode/GDB Stub Mode** command. If configured in your project's `sdkconfig`, it can trigger the start of a debug session for GDB remote protocol server (GDBStub) or [ESP-IDF Core Dump](#) when an error occurs. For more information, see [Panic Handler](#).

Core Dump is configured when **Core Dump's Data Destination** is set to either UART or FLASH using the ESP-IDF: SDK Configuration Editor extension command or `idf.py menuconfig` in a terminal.

GDB Stub is configured when **Panic Handler Behaviour** is set to Invoke GDBStub using the ESP-IDF: SDK Configuration Editor extension command or `idf.py menuconfig` in a terminal.

ESP-IDF: Image Viewer

The ESP-IDF extension provides an **ESP-IDF: Image Viewer** feature that allows you to visualize binary image data from debug variables during a debugging session. This is particularly useful for applications that work with camera sensors, display buffers, LVGL graphics, OpenCV computer vision, or any raw image data.

Quick Access Methods:

Right-click on variables in the debug session: - Right-click on any image-related variable (`lv_image_dsc_t`, `cv::Mat`, `png_image`, etc.) and select `View Variable as Image` - The Image Viewer automatically detects the variable type and extracts the appropriate image properties

Manual Image Viewer: - Go to `View > Command Palette` and enter `ESP-IDF: Open Image Viewer` - Enter the name of your image data variable and its size - Select the appropriate image format and

dimensions - Click Load Image to visualize the data

Supported Image Formats:

The Image Viewer supports a comprehensive range of image formats:

RGB Formats: - RGB565, RGB888, RGBA8888, ARGB8888, XRGB8888 - BGR888, BGRA8888, ABGR8888, XBGR8888 - RGB332, RGB444, RGB555, RGB666, RGB777 - RGB101010, RGB121212, RGB161616

Other Formats: - Grayscale (8-bit per pixel) - YUV420, YUV422, YUV444 (various YUV formats)

Built-in Support:

LVGL Image Descriptor (lv_image_dsc_t): - Automatically extracts format, dimensions, and data from LVGL structures - Supports all LVGL color formats with automatic mapping to display formats

OpenCV Mat (cv::Mat): - Automatically extracts dimensions, format, and data from OpenCV Mat objects - Supports BGR888, BGRA8888, and Grayscale formats

Example Usage:

LVGL Image Example:

```
// LVGL image descriptor
lv_image_dsc_t my_image = {
    .header = {
        .cf = LV_COLOR_FORMAT_RGB888, // Color format
        .w = 320,                      // Width
        .h = 240                      // Height
    },
    .data_size = 320 * 240 * 3,        // Data size in bytes
    .data = image_data                // Pointer to image data
};
```

During debugging, right-click on my_image and select View Variable as Image. The Image Viewer will automatically detect it as an LVGL image and extract the format, dimensions, and data.

OpenCV Mat Example:

```
cv::Mat image(240, 320, CV_8UC3); // 320x240 BGR888 image
// ... populate image data ...
```

During debugging, right-click on image and select View Variable as Image. The Image Viewer will automatically detect it as an OpenCV Mat and extract the dimensions, format, and data.

Manual Raw Data Example:

```
uint8_t image_buffer[320 * 240 * 3]; // RGB888 format, 320x240 pixels
size_t image_size = sizeof(image_buffer);
```

For manual usage: - Enter `image_buffer` as the variable name - Enter `image_size` or `230400` (`320 * 240 * 3`) as the size - Select `RGB888` format - Set width to `320` and height to `240`

Custom Image Format Configuration:

You can extend the Image Viewer to support custom image formats by creating a JSON configuration file and setting the `idf.imageViewerConfigs` configuration option.

Example Custom Configuration:

```
[
  {
    "name": "Custom Image Structure",
    "typePattern": "my_image_t",
    "width": {
      "type": "string",
      "isChild": true,
      "value": "w"
    },
    "height": {
      "type": "string",
      "isChild": true,
      "value": "h"
    },
    "format": {
      "type": "number",
      "isChild": false,
      "value": "0x0E"
    },
    "dataAddress": {
      "type": "string",
      "isChild": true,
      "value": "pixels"
    },
    "dataSize": {
      "type": "formula",
      "isChild": false,
      "value": "$var.w * $var.h * 3"
    },
    "imageFormats": {
      "14": "rgb888",
      "15": "rgba8888"
    }
  }
]
```

Configuration Options:

typePattern: Regex pattern to match the GDB type of the selected variable when right-clicking **View Variable as Image** (e.g., `"my_image_t"`, `"lv_image_dsc_t"`, `"cv::Mat|Mat"`)

width/height: Configuration for extracting image dimensions

format: Configuration for extracting image format

dataAddress: Configuration for extracting image data pointer

dataSize: Configuration for calculating image data size (supports formulas)

imageFormats: Mapping of numeric format values to display format strings

Field Configuration Details:

Each field (width, height, format, dataAddress, dataSize) has the following properties:

type: Specifies the data type of the field value: - "string": The value is a string (field name or expression) - "number": The value is a numeric constant (e.g., "0x0E", "14") - "formula": The value is a mathematical formula (only for dataSize field)

isChild: Determines how the field value is interpreted: - true: The value represents a child field of the right-clicked variable (e.g., "header.w", "data") - false: The value is a direct expression or constant that can be evaluated by GDB

value: The actual field name, expression, or constant to use for extraction

Important Configuration Notes:

dataSize Formula: When using formulas in the dataSize field, the string \$var will be automatically replaced with the actual variable name when you right-click and select **View Variable as Image**. For example, if your variable is named my_image and the formula is \$var.w * \$var.h * 3, it will be evaluated as my_image.w * my_image.h * 3. **Note:** The formula must be a valid GDB expression since it is calculated by GDB itself.

Format Number Mapping: The numeric keys in the imageFormats object must match the actual numeric values that the format field extracts from your image structure. For example, if your image structure's format field contains the value 14, then the imageFormats object should have a key "14" that maps to the appropriate display format string like "rgb888".

Important Notes: - **Automatic Detection:** The Image Viewer automatically detects supported image types and extracts properties - **Unified Interface:** Single View Variable as Image command works for all supported formats - **Format Validation:** All formats are validated against supported display formats - **Raw Data:** The Image Viewer supports raw pixel formats. Compressed formats (JPEG, PNG, etc.) are not supported - **Size Specification:** For manual usage, you must specify the correct size of the image data array - **Variable Size:** The size can be provided as a number (bytes) or as the name of another variable containing the size - **Pointer Variables:** For pointer variables, make sure to provide the actual data size, not the pointer size - **Auto-Dimensioning:** The Image Viewer automatically estimates dimensions based on the data size and selected format, but you can manually adjust them for better results - **Extensible:** Custom image formats can be added through configuration files

Other extensions debug configuration

While we support the ESP-IDF extension, you can also use other extensions to debug your project. For example, you can use the [Microsoft C/C++](#) extension to debug your project.

To do this, you need to configure the launch.json file in the .vscode directory of your project. Here is an example of a launch.json file:

```
{
  "configurations": [
    {
      "name": "GDB",
      "type": "cppdbg",
      "request": "launch",
      "MIMode": "gdb",
      "miDebuggerPath": "${command:espIdf.getToolchainGdb}",
      "miDebuggerServerAddress": "localhost:3333",
      "program": "${workspaceFolder}/build/${command:espIdf.getProjectName}.elf",
      "windows": {
        "program": "${workspaceFolder}\\build\\${command:espIdf.getProjectName}.elf"
      },
      "cwd": "${workspaceFolder}",
      "environment": [
        {
          "name": "KEY",
          "value": "VALUE"
        }
      ],
      "setupCommands": [
        {
          "text": "set remotetimeout 20"
        }
      ],
      "postRemoteConnectCommands": [
        {
          "text": "mon reset halt"
        },
        {
          "text": "maintenance flush register-cache"
        }
      ],
      "externalConsole": false,
      "logging": {
        "engineLogging": true
      }
    }
  ]
}
```

Another recommended debug extension is the [Native Debug](#) extension. Here is an example configuration for the launch.json file:

```
{
  "configurations": [
    {
```

```
    "type": "gdb",
    "request": "attach",
    "name": "NativeDebug",
    "target": "target remote localhost:3333",
    "executable": "${workspaceFolder}/build/${command:espIdf.getProjectName}.elf",
    "gdbpath": "${command:espIdf.getToolchainGdb}",
    "cwd": "${workspaceRoot}",
    "autorun": [
        "mon reset halt",
        "maintenance flush register-cache",
        "thb app_main"
    ]
  }
}
```

Consider that if you use these extension debugger configuration you need to manually run OpenOCD from ESP-IDF VS Code extension [OpenOCD] status bar button or from terminal.

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Additional IDE Features

[ESP-IDF Extension for VSCode](#)

- »
 - Additional IDE Features
 - [Edit on GitHub](#)
-

Additional IDE Features

[ä.æ]

- [Application Size Analysis](#)
- [Application Tracing](#)
- [CMakeLists.txt Editor](#)
- [Code Coverage](#)
- [Device Firmware Upgrade \(DFU\) via USB](#)
- [Docker Container](#)
- [ESP-IDF Terminal](#)
- [ESP-IDF Profiles](#)
- [eFuse Explorer](#)
- [Flash Encryption](#)
- [Heap Tracing](#)
- [Hints Viewer](#)
- [Install ESP-IDF Components](#)
- [Language Tools](#)
- [NVS Partition Table Editor](#)
- [Partition Table Editor](#)
- [Project Configuration Editor](#)
- [QEMU Integration](#)
- [ESP RainMaker](#)
- [Unit Testing with Unity](#)
- [Working with Multiple Projects](#)
- [Using WSL in Windows](#)
- [Web Extension](#)

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Troubleshooting

[ESP-IDF Extension for VSCode](#)

- »
 - Troubleshooting
 - [Edit on GitHub](#)
-

Troubleshooting

[\[æ\]](#)

Note

Set `idf.openOcdDebugLevel` to 4 or higher in your `<project-directory>/vscode/settings.json` to enable debug level logs of OpenOCD server in ESP-IDF output.

Set `verbose: true` in your `<project-directory>/vscode/launch.json` to display more debug adapter output.

In Visual Studio Code, go to `View > Output` and select ESP-IDF from the dropdown. This output provides useful information about the extension's activity.

In Visual Studio Code, go to `View > Command Palette...` and type `ESP-IDF: Doctor Command` to generate a report of your environment configuration. This report will be copied to your clipboard for easy pasting.

Check the log file located at:

Windows:

`%USERPROFILE%\vscode\extensions\espressif.esp-idf-extension-VERSION\esp_idf_vsc_ext.log`

macOS/Linux:

`$HOME/.vscode/extensions/espressif.esp-idf-extension-VERSION/esp_idf_vsc_ext.log`

In Visual Studio Code, go to `Help > Toggle Developer Tools` and copy any errors from the Console tab related to this extension.

In Visual Studio Code select menu `View > Output > Extension Host`. This output information is useful to know what is happening during the extensions activation. If no extension command work, you could share the output here to see the error stack.

Visual Studio Code allows you to configure settings at different levels: **Global (User Settings)**, **Workspace** and **Workspace Folder**. Ensure your project uses the correct settings. The output from `ESP-IDF: Doctor` command will show the settings currently in use.

Workspace folder configuration settings are defined in `${workspaceFolder}/.vscode/settings.json`

Workspace configuration settings are defined in the workspace's `<name>.code-workspace` file

User settings defined in `settings.json`

Windows: `%APPDATA%\Code\User\settings.json`

MacOS: `$HOME/Library/Application Support/Code/User/settings.json`

Linux: `$HOME/.config/Code/User/settings.json`

This extension uses the `idf.saveScope` configuration setting (which can only be defined in User Settings) to specify where to save settings for features such as the Setup Wizard. You can modify this using the ESP-IDF: Select where to Save Configuration Settings command.

Review [OpenOCD Troubleshooting FAQ](#) for information related to OpenOCD output, application tracing, debugging, or any OpenOCD issues.

Note

If you receive errors like `unable to create symlink` while cloning ESP-IDF on Windows, enabling **Developer Mode** may help resolve the issue.

EIM Launch Modes (GUI vs CLI)

When running the **ESP-IDF: Open ESP-IDF Installation Manager** command, the extension now selects the launch mode automatically:

If an existing EIM installation is found and it supports the `gui` subcommand, the extension opens the EIM graphical application.

If the detected EIM only supports CLI commands, the extension runs EIM in CLI mode inside the VS Code integrated terminal.

If EIM is not yet installed, the extension downloads the GUI-capable build in local desktop environments and the CLI build in remote or headless environments.

The extension updates the `idf.eimExecutableArgs` setting automatically based on the mode it launches.

Remote and Headless Environments

In remote environments (SSH, WSL, Dev Containers, Codespaces) and browser-based VS Code, the GUI cannot be displayed. The extension automatically forces CLI (wizard) mode in these cases and no prompt is shown.

For Linux-based remote users, the extension also appends the EIM executable directory to the user's shell PATH the first time it launches `eim wizard`. This allows running `eim` directly from future terminals without copying the full binary path.

After opening a new terminal, you can use the [EIM CLI Commands](#) reference to run commands such as `eim list` or `eim run` yourself.

VS Code Installed via Snap (Ubuntu)

If you installed VS Code via snap (the default method on Ubuntu), the EIM GUI cannot be launched due to sandbox restrictions. You may see the following error:

```
The terminal process "/usr/bin/bash" terminated with exit code: 127.
```

When snap is detected and the resolved EIM supports the GUI, the extension shows a modal with two options:

Run EIM in Terminal  launches EIM in CLI mode directly in the VS Code integrated terminal.

Copy EIM Path  copies the EIM binary path to your clipboard so you can run the GUI manually from a system terminal (e.g., GNOME Terminal, Konsole).

The default EIM path on Linux is:

```
~/espressif/eim_gui/eim
```

Alternatively, you can **install VS Code via the .deb package** (recommended) to remove snap's sandbox restrictions entirely:

```
sudo snap remove code
# Then install the .deb package downloaded from https://code.visualstudio.com/Download
```

If you cannot resolve the error, please search the [GitHub Repository Issues](#) for existing issues or create a new issue [here](#).

EIM Path Resolution Order

When the extension needs to locate the EIM binary, it checks the following locations in order, stopping at the first match:

System PATH  runs `which eim` / `where eim` to find an EIM already on the PATH.

eim_idf.json  reads the `eimPath` field from the JSON file at the path configured by `idf.eimIdfJsonPath`. Default locations:

Windows: `C:\Espressif\tools\eim_idf.json`

macOS/Linux: `$HOME/.espressif/tools/eim_idf.json`

EIM_PATH environment variable  reads the `EIM_PATH` environment variable.

Managed install directory checks the extension-managed install folders. In normal desktop environments the GUI build is preferred; in headless, remote, or snap environments the CLI build is checked first.

If the extension fails to find or launch EIM, run the **ESP-IDF: Doctor Command** and inspect the `esp_idf_vsc_ext.log` file. Each step above is logged with the path being checked, so you can identify exactly which location is missing or incorrect.

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Settings

[ESP-IDF Extension for VSCode](#)

- »
 - ESP-IDF Settings
 - [Edit on GitHub](#)
-

ESP-IDF Settings

[\[ä,æ](#)

Visual Studio Code allows you to configure settings at different levels: **Global (User Settings)**, **Workspace** and **Workspace Folder**. Ensure your project uses the correct settings.

Workspace folder configuration settings are defined in `${workspaceFolder}/.vscode/settings.json`

Workspace configuration settings are defined in the workspace's `<name>.code-workspace` file

User settings defined in `settings.json`

Windows: `%APPDATA%\Code\User\settings.json`

MacOS: `$HOME/Library/Application Support/Code/User/settings.json`

Linux: `$HOME/.config/Code/User/settings.json`

This extension uses the **idf.saveScope** configuration setting (which can only be defined in User Settings) to specify where to save settings for features such as the Setup Wizard. You can modify this using the ESP-IDF: Select where to Save Configuration Settings command.

This extension contributes the following settings that can be later updated in `settings.json` or from VS Code Settings Preference Menu by:

Navigate to **View > Command Palette**.

Search for **Preferences: Open User Settings (JSON)**, **Preferences: Open Workspace Settings (JSON)** or **Preferences: Open Settings (UI)** and select the command to open the setting management window.

Note

Please note that `~`, `%VARIABLE%` and `$VARIABLE` are not recognized when set on ANY of this extension configuration settings. Instead, you can set any environment variable in the path using `${env:VARIABLE}`, such as `${env:HOME}`, or refer to other configuration parameter path with `${config:SETTINGID}`, such as

`${config:idf.buildPath}`.

Note

All settings can be applied to Global (User Settings), Workspace and WorkspaceFolder unless Scope is specified.

The settings are grouped in the same categories as in the VS Code Settings UI (and in `package.json`):

Setup & Installation

Setting ID	Description
idf.eimIdfJsonPath	Path to the ESP-IDF Installation Manager (EIM) <code>eim_idf.json</code> file. Scope: Application.
idf.eimExecutableArgs	Arguments for the EIM executable (default: <code>["gui", "--idf-features ide"]</code>). This value is updated automatically by the extension based on the detected EIM launch mode. Scope: Application.
idf.currentSetup	Path to the currently selected ESP-IDF directory. Scope: Resource.
idf.saveLastProjectConfiguration	Restore the project configuration that was selected last when the workspace is reopened (default: <code>true</code>). When disabled, no configuration is selected on startup. Scope: Resource.
idf.gitPath	Path to the Git executable (default: <code>/usr/bin/git</code>). Scope: Application.
idf.gitPathWin	Path to the Git executable in Windows (default: <code>\${env:programfiles}\\Git\\cmd\\git.exe</code>). Scope: Application.
idf.extensionActivationMode	Controls extension activation mode: "detect" (default), "always", or "never".

idf.gitPath (or **idf.gitPathWin** on Windows) is used by the extension to clone ESP-IDF or additional supported frameworks such as ESP-ADF, ESP-MDF and Arduino-ESP32.

Serial Port

Setting ID	Description
idf.port	Path of selected device port (default: <code>detect</code>). Scope: Resource.

Setting ID	Description
idf.portWin	Path of selected device port in Windows (default: detect). Scope: Resource.
idf.monitorPort	Optional path of selected device port for monitor. If undefined, idf.port is used. Scope: Resource.
idf.flashBaudRate	Flash baud rate (default: 460800). Used for ESP-IDF: Flash your Project and debugging. Scope: Resource.
idf.monitorBaudRate	Monitor baud rate. Empty by default to use sdkconfig CONFIG_ESP_CONSOLE_UART_BAUDRATE. Scope: Resource.
idf.enableSerialPortChipIdRequest	Enable detecting the chip ID and show on serial port selection list. Scope: Application.
idf.useSerialPortVendorProductFilter	Enable use of idf.usbSerialPortFilters to filter serial port devices list. Scope: Application.
idf.usbSerialPortFilters	USB productId and vendorId map to filter known Espressif devices. Scope: Application.
idf.serialPortDetectionTimeout	Timeout in seconds for serial port detection using esptool.py (default: 60). Scope: Resource.

Flash

Setting ID	Description
idf.openOcdConfigs	Configuration files for OpenOCD, relative to OPENOCD_SCRIPTS folder. If idf.openOcdLaunchArgs is defined this setting is ignored. Scope: Resource.
idf.flashType	Preferred flash method: UART, JTAG or DFU. Scope: Resource.
idf.partitionToUse	Partition to use during build and flash, including Build, Flash and Monitor (default: all). Options: all, app, bootloader, partition-table. Scope: Resource.
idf.jtagFlashCommandExtraArgs	OpenOCD JTAG flash extra arguments (default: ["verify", "compress", "reset"]). Scope: Resource.

Setting ID	Description
idf.preFlashTask	Command string to execute before flash task. Scope: Resource.
idf.postFlashTask	Command string to execute after flash task. Scope: Resource.

Note

When using **ESP-IDF: Set Espressif Device Target**, the extension overrides the current sdkconfig `IDF_TARGET` and updates **idf.openOcdConfigs** with default OpenOCD config files. To customize **idf.openOcdConfigs** only, use **ESP-IDF: Select OpenOCD Board Configuration** or edit `settings.json`.

Build

Setting ID	Description
idf.buildPath	Custom build directory for extension commands (default: <code>\${workspaceFolder}/build</code>). Scope: Resource.
idf.buildPathWin	Custom build directory in Windows (default: <code>\${workspaceFolder}\\build</code>). Scope: Resource.
idf.cmakeCompilerArgs	Arguments for CMake configuration (default includes <code>-G Ninja</code> , <code>-DPYTHON_DEPS_CHECKED=1</code> , <code>-DESP_PLATFORM=1</code>). Scope: Resource.
idf.sdkconfigDefaults	List of sdkconfig default values for initial build configuration. Scope: Resource.
idf.ninjaArgs	Arguments for Ninja build task. Scope: Resource.
idf.customExtraVars	Key-value object of environment variables added to extension command processes (e.g. <code>OPENOCD_SCRIPTS</code>). Does not modify system environment outside VS Code. Scope: Resource.
idf.useIDFKconfigStyle	Enable style validation for Kconfig files. Scope: Resource.
idf.saveBeforeBuild	Save all edited files before building (default: <code>true</code>). Scope: Resource.
idf.enableCCache	Enable CCache in build task (ensure CCache is in <code>PATH</code>). Scope: Resource.

Setting ID	Description
idf.extraCleanPaths	Additional paths to delete on Full Clean Project (default: []). E.g. ["managed_components", "dependencies.lock"]. Scope: Resource.
idf.sdkconfigFilePath	Absolute path for the sdkconfig file. Scope: Resource.
idf.enableSizeTaskAfterBuildTask	Run IDF Size task after IDF Build task (default: true). Scope: Resource.
idf.preBuildTask	Command string to execute before build task. Scope: Resource.
idf.postBuildTask	Command string to execute after build task. Scope: Resource.

Debug & OpenOCD

Setting ID	Description
openocd.tcl.host	OpenOCD TCL server host (default: localhost). Scope: Resource.
openocd.tcl.port	OpenOCD TCL server port (default: 6666). Scope: Resource.
openocd.jtag.command.force_unix_path_separator	Use / instead of \ as path separator on Win32 (default: true).
idf.launchMonitorOnDebugSession	Launch ESP-IDF Monitor when starting an ESP-IDF debug session (default: false). Scope: Resource.
idf.openOcdDebugLevel	OpenOCD debug level 0-4 (default: 2). 0 = error only, 4 = verbose. Ignored if idf.openOcdLaunchArgs is set. Scope: Resource.
idf.customOpenOCDPath	Absolute path to a custom OpenOCD executable. If empty or the path does not exist, the extension falls back to searching OpenOCD in PATH. Scope: Resource.
idf.openOcdLaunchArgs	Custom arguments for OpenOCD. If set, idf.openOcdConfigs and idf.openOcdDebugLevel are ignored. Command form otherwise: openocd -d\${idf.openOcdDebugLevel} -f \${idf.openOcdConfigs}. Scope: Resource.

Setting ID	Description
<code>idf.svdFilePath</code>	SVD file path for chip debug peripheral tree view (default: <code>\${workspaceFolder}/esp32.svd</code>). Scope: Resource.

Monitori

Setting ID	Description
<code>idf.monitorDelay</code>	Delay in milliseconds before starting debug session after IDF Monitor or when breaking monitor (default: 1000). Scope: Resource.
<code>idf.monitorNoReset</code>	Enable no-reset flag for IDF Monitor (default: <code>false</code>). Scope: Resource.
<code>idf.monitorEnableTimestamps</code>	Enable timestamps in IDF Monitor (default: <code>false</code>). Scope: Resource.
<code>idf.monitorCustomTimestampFormat</code>	Custom timestamp format for IDF Monitor. Scope: Resource.

App Tracei

Setting ID	Description
<code>trace.poll_period</code>	Poll period for apptrace. Scope: Resource.
<code>trace.trace_size</code>	Trace size for apptrace (default: 2048). Scope: Resource.
<code>trace.stop_tmo</code>	Stop timeout for apptrace (default: 3). Scope: Resource.
<code>trace.wait4halt</code>	Wait for halt for apptrace (0 or 1, default: 0). Scope: Resource.
<code>trace.skip_size</code>	Skip size for apptrace (default: 0). Scope: Resource.

Tasks & Terminali

Setting ID	Description
<code>idf.customTask</code>	Custom task command for ESP-IDF: Execute Custom Task . Scope: Resource.

Setting ID	Description
idf.customTerminalExecutable	Absolute path to shell executable for extension terminals (default: VS Code default). Scope: Resource.
idf.customTerminalExecutableArgs	Arguments for idf.customTerminalExecutable . Scope: Resource.

UI & Notifications

Setting ID	Description
idf.showOnboardingOnInit	Show ESP-IDF configuration/onboarding window on extension activation (default: true). Scope: Window.
idf.hasWalkthroughBeenShown	Whether the walkthrough has been shown. Scope: Application.
idf.notificationMode	Notifications and output focus mode: Silent, Notifications, Output, All (default: All). Scope: Resource.
idf.saveScope	Where to save extension settings: Global (1), Workspace (2), WorkspaceFolder (3). Scope: Application only.
idf.enableStatusBar	Show or hide extension status bar items (default: true). Scope: Resource.
idf.enableUpdateSrcsToCMakeListsFile	Enable updating source files in CMakeLists.txt (default: true). Scope: Window.

Telemetry

Setting ID	Description
idf.telemetry	Enable telemetry (default: true).

ESP Rainmaker Settings

Setting ID	Description
esp.rainmaker.api.server_url	ESP Rainmaker API server URL (default: https://api.rainmaker.espressif.com/v1/). Scope: Resource.

Setting ID	Description
esp.rainmaker.oauth.url	ESP Rainmaker OAuth URL (default: https://auth.rainmaker.espressif.com/oauth2/authorize). Scope: Resource.

Coveragei

Setting ID	Description
idf.coveredLightTheme	Background color for covered lines (light theme) in gcov coverage (default: <code>rgba(0, 128, 0, 0.1)</code>). Scope: Resource.
idf.coveredDarkTheme	Background color for covered lines (dark theme) in gcov coverage (default: <code>rgba(0, 128, 0, 0.1)</code>). Scope: Resource.
idf.partialLightTheme	Background color for partially covered lines (light theme) (default: <code>rgba(250, 218, 94, 0.1)</code>). Scope: Resource.
idf.partialDarkTheme	Background color for partially covered lines (dark theme) (default: <code>rgba(250, 218, 94, 0.1)</code>). Scope: Resource.
idf.uncoveredLightTheme	Background color for uncovered lines (light theme) (default: <code>rgba(255, 0, 0, 0.1)</code>). Scope: Resource.
idf.uncoveredDarkTheme	Background color for uncovered lines (dark theme) (default: <code>rgba(255, 0, 0, 0.1)</code>). Scope: Resource.

Component Manageri

Setting ID	Description
idf.enableIdfComponentManager	Enable IDF Component Manager in build command (default: <code>false</code>). Scope: Resource.
esp.component-manager.url	ESP Component Registry URL (default: https://components.espressif.com). Scope: Resource.

QEMU Settingsi

Setting ID	Description
idf.qemuDebugMonitor	Enable QEMU Monitor in debug session (default: true). Scope: Resource.
idf.qemuExtraArgs	Extra arguments for QEMU. Scope: Resource.

Unit Test

Setting ID	Description
idf.unitTestFilePattern	Glob pattern for unit test file discovery (default: <code>**/test/test_*.c</code>). Scope: Resource.

Other

Setting ID	Description
idf.wssPort	WebSocket server port (default: 49203, range 49152â65535). Scope: Resource.
idf.sbomFilePath	Path for ESP-IDF SBOM report (default: <code>espidf.spdx</code>). Scope: Resource.
idf.imageViewerConfigs	Path to custom image format configurations JSON for the Image Viewer (relative to workspace or absolute). Scope: Resource.

Use of Environment Variables in ESP-IDF settings.json and using other ESP-IDF settings within ESP-IDF settings

Environment (env) variables and other ESP-IDF settings (config) can be referenced in ESP-IDF settings using the syntax `${env:VARNAME}` and `${config:ESPIDFSETTING}`, respectively.

You can also prepend a string to the result of the other ESP-IDF settings (config) by using the syntax `${config:ESPIDFSETTING,prefix}`. The prefix will be added directly to the beginning of each string value. For example `${config:idf.openOcdConfigs,-f}` will add `-f` to the beginning of each string value of **idf.openOcdConfigs**. If `"idf.openOcdConfigs": ["interface/some.cfg", "target/some.cfg"]` the expansion will be `-finterface/some.cfg -ftarget/some.cfg`.

For example, to use `"~/workspace/blink"`, set the value to `"${env:HOME}/workspace/blink"`.

[Previous](#) [Next](#)

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

See all Commands

[ESP-IDF Extension for VSCode](#)

- »
- List of Available Commands
- [Edit on GitHub](#)

List of Available Commands

[ä.æ]

All commands start with ESP-IDF:.

Command	Description
Add Arduino ESP32 as ESP-IDF Component	Add Arduino-ESP32 as an ESP-IDF component in your current directory (<code>\${CURRENT_DIRECTORY}/components/arduino</code>).
Add Docker Container Configuration	Add the .devcontainer files to the currently opened project directory. It is necessary to use an ESP-IDF project in a Docker container with Visual Studio Code Dev Containers extension.
Add Editor Coverage	Parse your project GCOV Code coverage files to add color lines representing code coverage on currently opened source code file.
Add VS Code Configuration Folder	Add .vscode files to the currently opened project directory. This includes launch.json (for debugging), settings.json and c_cpp_properties.json for syntax highlight.
Build, Flash and Start a Monitor on Your Device	Build the project, write binaries program to device and start a monitor terminal with a single command. Similar to <code>idf.py build flash monitor</code> .
Build Your Project	Build your project using CMake and Ninja-build as explained in ESP-IDF Build System Using Cmake Directly . You can modify the behavior of the build task with idf.cmakeCompilerArgs for Cmake configuration step and idf.ninjaArgs for Ninja step. For example, using [-j N] where N is the number of jobs running in parallel.

Command	Description
Clear eFuse Summary	Clear the eFuse Summary tree from ESP Explorer EFUSEEXPLORER.
Clear ESP-IDF Search Results	Clear results from ESP Explorer Documentation Search Results.
Configure Project SDKConfig for Coverage	Set required values in your project SDKConfig to enable code coverage analysis.
Configure project for ESP-Clang	Configure the current ESP-IDF project to use esp-clang as toolchain to use and configure LLVM Clang extension in settings.json.
Create New ESP-IDF Component	Create a new component in the current directory based on ESP-IDF component template.
Create New Empty Project	Ask for the new project name, choose the directory to create the project, and show a notification to open the newly created project.
Create Project Configuration	Create CMakePresets.json in the project root with a starter default and production configure preset to rename and extend. Presets you already defined are left untouched.
Dispose of Current SDK Configuration Editor Server Process	If you already executed the SDK Configuration editor, a cache process will remain in the background for faster reopening. This command will dispose of such cache process.
Doctor Command	Run a diagnostic on the extension setup settings and logs to generate a troubleshooting report.
Troubleshoot Form	Launch UI for the user to submit a troubleshooting report with steps to reproduce the issue. Run a diagnostic on the extension setup settings and logs, and send the information to the telemetry backend.
Encrypt and Flash Your Project	Execute flashing the project program to device while adding encrypt for partitions to be encrypted.
Erase Flash Memory from Device	Execute esptool.py erase_flash command to erase flash chip (set to 0xFF bytes).
Execute Custom Task	User can define a command-line command or script in idf.customTask , which can be executed with this command.

Command	Description
Flash Your Project	Write binary data to the ESP8266's flash chip from your current ESP-IDF project. This command will use either UART, DFU or JTAG based on <code>idf.flashType</code> .
Flash (DFU) your project	Write binary data to the ESP8266's flash chip from your current ESP-IDF project using DFU. Only for ESP32-S2 and ESP32-S3.
Flash (UART) your project	Write binary data to the ESP8266's flash chip from your current ESP-IDF project using esptool.py
Flash (with JTag)	Write binary data to the ESP8266's flash chip from your current ESP-IDF project using OpenOCD JTAG
Full Clean Project	Delete the current ESP-IDF project build directory.
Get eFuse Summary	Retrieve a list of eFuses and their corresponding values from the chip currently connected to the serial port.
Get HTML Coverage Report for Project	Parse your project GCOV Code coverage files to generate an HTML coverage report.
Import ESP-IDF Project	Import an existing ESP-IDF project, add .vscode and .devcontainer files to a new location, and optionally rename the project.
Install ESP-ADF	Clone ESP-ADF inside the selected directory and set <code>idf.customExtraVars[ESP_ADF_PATH]</code> configuration setting.
Launch IDF Monitor for Core Dump Mode/GDB Stub Mode	Launch ESP-IDF Monitor with WebSocket capabilities. If you have configured the panic handler to gdbstub or core dump, the monitor will launch a post-mortem debug session of the chip.
Launch QEMU Server	As described in QEMU documentation , this command will execute ESP32 QEMU from the project Dockerfile with the current project binaries.
Launch QEMU Debug Session	As described in QEMU documentation , this command will start a debug session to ESP32 QEMU from the project Dockerfile with the current project binaries.
Monitor Device	This command will execute idf.py monitor to start serial communication with Espressif device. Please take a look at the IDF Monitor Documentation .

Command	Description
Monitor QEMU Device	As described in QEMU documentation , this command will start a terminal to monitor the ESP32 QEMU from the project Dockerfile with the current project binaries.
New Project	Launch UI with an ESP-IDF project creation wizard using examples templates from ESP-IDF and ESP-ADF.
NVS Partition Editor	Launch UI to create a CSV file for ESP-IDF Non-Volatile Storage Library .
Open ESP-IDF Terminal	Open a terminal with IDF_PATH and Python virtual environment activated.
OpenOCD Adaptor (serial & location)	Toggle the status bar item that shows the OpenOCD adapter serial number (S) and USB adapter location (L), useful to verify which adapter will be used when multiple devices are connected.
Open ESP-IDF Install Manager	Open the ESP-IDF Install Manager to install and manage ESP-IDF versions and tools.
Partition Table Editor	Launch UI to manage custom partition table as described in ESP-IDF Partition Tables .
Pick a Workspace Folder	When using a Visual Studio Code workspace with multiple workspace folders, this command allows you to choose which workspace folder to apply this extension's commands to. More information can be found in working with multiple projects .
Remove Editor Coverage	Remove editor colored lines from Add Editor Coverage command.
Run idf.py reconfigure Task	This command will execute idf.py reconfigure (CMake configure task), which is useful for generating compile_commands.json for the C/C++ language support.
Run ESP-IDF-SBOM Vulnerability Check	Create software bill of materials (SBOM) files in the Software Package Data Exchange (SPDX) format for applications generated by the Espressif IoT Development Framework (ESP-IDF).
Save Default SDKCONFIG File (save-defconfig)	Generate sdkconfig.defaults files using the project current sdkconfig file.
SDK Configuration Editor	Launch UI to configure your ESP-IDF project settings. This is equivalent to idf.py menuconfig .

Command	Description
Search in documentationâ	Select some text from your source code file and search in ESP-IDF documentation with results right in the VS Code ESP-IDF Explorer tab.
Search Error Hint	Type some text to find a matching error from ESP-IDF hints dictionary.
Select Flash Method	Select which flash method to use for Flash Your Project command. It can be DFU, JTAG or UART.
Select Port to Use	Select which serial port to use for ESP-IDF tasks, such as flashing or monitoring your device.
Select OpenOCD Board Configuration	Select the OpenOCD configuration files that match your Espressif device target, such as DevKitC or ESP-Wrover-Kit. This is necessary for flashing with JTAG or debugging your device.
Select Project Configuration	Choose which configure preset from CMakePresets.json or CMakeUserPresets.json overrides the extension settings. Presets marked hidden and presets disabled by their condition are not offered.
Select Where to Save Configuration Settings	In Visual Studio Code, settings can be saved in three places: User Settings (global settings), workspace (.code-workspace file) or workspace folder (.vscode/settings.json).
Select Output and Notification Mode	This extension shows many notifications and output in the Output window ESP-IDF . This command allows you to set if to show notifications only, output only, both notifications and output, or neither.
Set Espressif Device Target	This will set the target for the current project (IDF_TARGET). Similar to idf.py set-target . For example, if you want to use ESP32 or ESP32-C3, you need to execute this command.
Show Ninja Build Summary	Execute the Chromium ninja-build-summary.py.
Size Analysis of the Binaries	Launch UI with the ESP-IDF project binaries size information.
Unit Test: Build Unit Test App	Build the unit test application for the current project. More information can be found in Unit Testing Documentation .

Command	Description
Unit Test: Flash Unit Test App	Flash the unit test application for the current project to the connected device. More information can be found in Unit Testing Documentation .
Unit Test: Build and Flash Unit Test App for Testing	Copy the unit test app in the current project, build the current project and flash the unit test application to the connected device. More information can be found in Unit Testing Documentation .
Load Image from LVGL C File	Load and display an image from a LVGL C file containing lv_image_dsc_t structure. This command allows you to view LVGL images without requiring a debug session.
Open Image Viewer	Open the Image Viewer panel to display images from debug variables or LVGL C files. This panel provides tools for viewing and analyzing image data in various formats.

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Extension Activation

[ESP-IDF Extension for VSCode](#)

- »
 - Extension Activation
 - [Edit on GitHub](#)
-

Extension Activation

This document explains how the ESP-IDF extension decides whether to activate when you open a workspace in Visual Studio Code.

How VS Code Extension Activation Works

Extension activation in VS Code is a **two-phase process**. Understanding both phases is essential to avoid confusion when the extension does not activate as expected.

Phase 1: VS Code Decides Whether to Load the Extension

Before the extension's own code can run, VS Code itself must decide to load it. This decision is based on **activation events** declared in the extension's `package.json`. The ESP-IDF extension registers the following activation triggers:

`workspaceContains: **/CMakeLists.txt` ■ VS Code loads the extension automatically when any `CMakeLists.txt` file exists anywhere in the workspace.

`onCommand: espIdf.*` ■ VS Code loads the extension when you run any ESP-IDF command from the Command Palette (e.g., *ESP-IDF: Build your Project*, *ESP-IDF: Flash your Project*).

`onView: idfPartitionExplorer`, `onView: espRainmaker`, etc. ■ VS Code loads the extension when you open one of its registered sidebar views.

`onLanguageModelTool: espIdfCommands` ■ VS Code loads the extension when a language-model integration (e.g., Copilot) invokes the ESP-IDF commands tool.

`contributes.mcpServerDefinitionProviders (espIdf.mcpServers)` ■ VS Code can load the extension when Chat discovers MCP servers. The Espressif Documentation and ESP Component Registry servers are registered at this point unless `idf.extensionActivationMode` is `never`.

If **none** of these triggers fire, VS Code will never load the extension and its `activate()` function will never run. This means:

Important

The `idf.extensionActivationMode` setting has **no effect** unless VS Code loads the extension first. If none of the activation events listed above fire (no `CMakeLists.txt` in the workspace, no ESP-IDF command run, no sidebar view opened, no language-model tool invocation, no MCP server discovery), the extension will not activate **â€”** even if `idf.extensionActivationMode` is set to "always".

Phase 2: Extension Decides Whether to Fully Initialize

Once VS Code loads the extension (Phase 1), the extension's `activate()` function runs. At this point, the extension reads the `idf.extensionActivationMode` setting and applies the priority hierarchy described in the next section to decide whether to proceed with full initialization or exit early.

Two-Phase Summary

Phase	Controlled By	What Happens
Phase 1 â€” Loading	VS Code platform (<code>activationEvents</code> in <code>package.json</code>)	VS Code decides whether to load and start the extension's code
Phase 2 â€” Initialization	Extension code (<code>idf.extensionActivationMode</code> setting)	Extension decides whether to fully initialize or exit early

Activation Mode Priority Hierarchy (Phase 2)

Once the extension is loaded by VS Code, it follows a strict priority hierarchy when deciding whether to fully initialize:

Workspace/Global Setting = **â€”alwaysâ€”**

- Action:** Initialize immediately
- Skips:** All other checks (CMake detection, folder checks)
- Use case:** Force-enable the extension for custom project layouts
- Setting location:** User Settings or Workspace Settings

Workspace/Global Setting = **â€”neverâ€”**

- Action:** Do NOT initialize, exit immediately
- Overrides:** All folder-level settings
- No prompt shown:** Respects your explicit choice
- Use case:** Explicitly disable extension in specific workspaces

This still applies when VS Code loads the extension via MCP discovery with no workspace open; MCP servers are also not registered. Servers are withdrawn when mode is "never" or the extension deactivates; they are not a user `mcp.json` entry.

ANY Folder Setting = `"always"`

Action: Initialize immediately (`"true wins"` strategy)

Order independent: Checks all folders, not just the first

Use case: Multi-root workspace with at least one ESP-IDF project

Setting location: Folder Settings in `.vscode/settings.json`

ALL Folder Settings = `"never"`

Action: Do NOT initialize, exit immediately

No prompt shown: Respects explicit configuration

Use case: Multi-root workspace explicitly excluding ESP-IDF

Otherwise (Default Behavior `"detect"` CMake Detection)

Condition: No workspace/global override (i.e., `"detect"`), no folder set to `"always"`, and not all folders set to `"never"` — this includes mixed per-folder states such as some `"detect"` and some `"never"`

Action: Fallback to automatic `CMakeLists.txt` content detection

Detection: Searches for `include($ENV{IDF_PATH}/tools/cmake/project.cmake)`

If not found: Prompts with `"Activate Anyway"` dialog

Use case: Standard ESP-IDF projects (backward compatible)

Configuration Setting `idf.extensionActivationMode`

The setting that controls extension initialization (Phase 2) is:

```
{
  "idf.extensionActivationMode": "detect"
}
```

Possible values:

`"detect"` (default) — Auto-detect by inspecting `CMakeLists.txt` content.

`"always"` — Skip detection and initialize immediately.

`"never"` — Never initialize, even if an ESP-IDF project is detected.

Setting Scope:

User (Global):

Linux: ~/.config/Code/User/settings.json

macOS: ~/Library/Application Support/Code/User/settings.json

Windows: %APPDATA%\Code\User\settings.json

Workspace: .vscode/settings.json in workspace root

Folder: .vscode/settings.json in specific folder (multi-root workspaces)

Usage Examples

Force Enable for Custom Projects

If you have a custom CMake setup that doesn't use the standard ESP-IDF `project.cmake` include:

Workspace Settings (.vscode/settings.json):

```
{
  "idf.extensionActivationMode": "always"
}
```

This will initialize the extension regardless of your `CMakeLists.txt` content.

Note

This only works if at least one Phase 1 activation event fires â for example, a `CMakeLists.txt` file exists in the workspace, you run an ESP-IDF command from the Command Palette, or you open an ESP-IDF sidebar view.

Disable Extension in Specific Workspace

If you want to prevent the extension from initializing in a non-ESP-IDF workspace:

Workspace Settings (.vscode/settings.json):

```
{
  "idf.extensionActivationMode": "never"
}
```

The extension will not initialize, and no prompt will be shown.

Multi-Root Workspace with Mixed Projects

For a workspace with both ESP-IDF and non-ESP-IDF projects:

Workspace File (`my-workspace.code-workspace`):

```
{
  "folders": [
    { "path": "esp32-firmware" },
    { "path": "documentation" },
    { "path": "web-interface" }
  ]
}
```

Folder-level settings (each folder's `.vscode/settings.json`):

`esp32-firmware/.vscode/settings.json`:

```
{
  "idf.extensionActivationMode": "always"
}
```

`documentation/.vscode/settings.json`:

```
{
  "idf.extensionActivationMode": "never"
}
```

In this example, the extension will initialize because the `esp32-firmware` folder has "always" set (Priority #3: `always` wins).

Standard ESP-IDF Project (No Configuration Needed)

For standard ESP-IDF projects with the typical `CMakeLists.txt` structure:

CMakeLists.txt (project root):

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(my-project)
```

No setting needed — VS Code automatically loads the extension because `CMakeLists.txt` exists (Phase 1), and the extension detects the ESP-IDF project include (Phase 2, Priority #5).

Workspace Without CMakeLists.txt

If your workspace does **not** contain any `CMakeLists.txt` file (e.g., a documentation-only workspace where you want ESP-IDF tooling available):

Setting `"idf.extensionActivationMode": "always"` alone is **not enough** — VS Code will not load the extension because no activation trigger fires.

To activate the extension, manually run any ESP-IDF command from the Command Palette (`Ctrl+Shift+P` — type `ESP-IDF`). This triggers Phase 1 loading via the `onCommand` activation event,

and the "always" setting ensures Phase 2 initialization proceeds.

Common Activation Issues

Extension Not Activating

Symptoms: ESP-IDF commands not available, no extension features working.

Possible causes:

No activation trigger (Phase 1): No declared activation event has fired â the workspace contains no `CMakeLists.txt` file, no ESP-IDF command has been run, no sidebar view has been opened, etc.

Solution: Run any ESP-IDF command from the Command Palette to trigger loading, or create a `CMakeLists.txt` file. If you want automatic activation in the future, set `"idf.extensionActivationMode": "always"` so that once loaded, it always initializes.

Explicit âneverâ setting (Phase 2): The extension is loaded but `idf.extensionActivationMode` is set to "never" in User, Workspace, or Folder settings.

Solution: Set it to "always" or "detect".

Non-standard CMakeLists.txt (Phase 2): Your project has a `CMakeLists.txt` but it doesnât include the standard ESP-IDF `project.cmake` line.

Solution: Add `"idf.extensionActivationMode": "always"` to workspace settings.

Prompt dismissed (Phase 2): You dismissed the âActivate Anywayâ dialog.

Solution: Reload the window (`Ctrl+Shift+P` â âDeveloper: Reload Windowâ) and choose âActivate Anywayâ when prompted.

Extension Activating in Wrong Workspace

Symptoms: ESP-IDF extension activated in a non-ESP-IDF project.

Why this happens: The workspace contains a `CMakeLists.txt` file (perhaps for a non-ESP-IDF C/C++ project), which triggers Phase 1 loading. The extension then either detects an ESP-IDF include or prompts the user.

Solution: Add `"idf.extensionActivationMode": "never"` to workspace settings to explicitly disable Phase 2 initialization.

Multi-Root Workspace Issues

Symptoms: Extension not activating even though one folder is an ESP-IDF project.

Possible cause: A workspace-level "never" setting is overriding folder settings.

Solution: Remove the workspace-level `idf.extensionActivationMode` setting and use folder-level settings instead.

Technical Details

Phase 1: Activation Events

The extension's `package.json` declares the following activation events:

workspaceContains:/CMakeLists.txt:** Fires when any folder in the workspace contains a `CMakeLists.txt` file at any depth. This is the most common automatic trigger.

onCommand:espIdf.*: Each ESP-IDF command is registered as an activation trigger. Running any command from the Command Palette will load the extension.

onView:*: Opening ESP-IDF sidebar panels (App Tracer, Partition Explorer, Rainmaker, Components) triggers loading.

onLanguageModelTool:espIdfCommands: Fires when a language-model integration (e.g., Copilot) invokes the ESP-IDF commands tool, enabling AI-assisted workflows.

These events are defined by the [VS Code Extension API](#) and cannot be changed via user settings. The only way to prevent Phase 1 loading is to disable the extension entirely in VS Code's Extensions view.

The extension also declares `contributes.mcpServerDefinitionProviders (espIdf.mcpServers)`. This is a contribution point, not an `activationEvents` entry. VS Code can still load the extension when Chat discovers the contributed MCP servers. The servers are withdrawn when `idf.extensionActivationMode` is "never" or the extension deactivates; they are not stored in a user `mcp.json` file.

Why "True Wins" Strategy?

In multi-root workspaces, if any folder needs the ESP-IDF extension, the extension must activate globally (VS Code extensions activate per workspace, not per folder). The "true wins" strategy ensures:

One ESP-IDF project folder can activate the extension for all folders

Documentation or utility folders can coexist without blocking activation

Better user experience in mixed-project workspaces

Backward Compatibility

The activation system maintains full backward compatibility:

Projects without the setting use the original `CMakeLists.txt` detection

The "Activate Anyway" prompt still appears for non-standard projects

Standard ESP-IDF projects work without any configuration changes

Performance Optimizations

The Phase 2 logic is designed for optimal performance:

Early Exit on Global Never: If workspace/global setting is "never", the extension exits immediately without checking folders or reading files.

Early Exit on Any Always: When any folder has "always", the extension stops checking remaining folders.

Skip CMake Detection: If any explicit "always" is found, CMake file detection is entirely skipped.

Lazy File Reading: CMakeLists.txt files are only read when the detection fallback is reached (no explicit "always" or all-"never" overrides).

See Also

[Troubleshooting](#)

[Settings](#)

[FAQs](#)

[VS Code Activation Events Documentation](#)

[GitHub Issue #1756](#)

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

FAQs

[ESP-IDF Extension for VSCode](#)

- »
 - FAQs
 - [Edit on GitHub](#)
-

FAQs

I have an issue or error with the Visual Studio Code extension. What can I do?

Refer to the [Troubleshooting documentation](#) to identify the error in the extension. You might resolve the issue yourself or provide enough information for others to assist you.

Search the [ESP-IDF extension repository](#) for Visual Studio Code first, and then check the forum. Your issue might already be resolved.

What about questions regarding ESP-IDF?

Consult [ESP-FAQ](#) or [ESP-IDF forum](#).

I try to build my project, but there is an error, and I don't know what is happening. What should I do?

Ensure you have configured the IDE plugin/extension properly. Review the documentation to [Install ESP-IDF and Tools](#).

The error might be in the setup or your project code. Refer to the [Troubleshooting documentation](#) and check for errors in these files.

Your issue might have been posted before in the [ESP-IDF GitHub repository](#) or [ESP-IDF forum](#). If not, you can open a new GitHub issue or start a topic in the forum.

How does an ESP-IDF project look?

Review the [Example Project](#) to understand how to use each file.

```
- myProject/  
  - CMakeLists.txt  
  - sdkconfig  
  - components/  
    - component1/  
      - CMakeLists.txt  
      - Kconfig  
      - src1.c  
    - component2/  
      - CMakeLists.txt  
      - Kconfig  
      - src1.c  
      - include/  
        - component2.h  
  - main/  
    - CMakeLists.txt  
    - src1.c  
    - src2.c
```

I tried flashing my project but encountered an error similar to `âError: unable to open ftdi device with vid.â` What should I do?[i](#)

Check the [Configure JTAG Interface](#) to ensure your drivers are correctly defined.

On Windows, try using [idf-env](#) by running the following command in a PowerShell terminal:

```
Invoke-WebRequest 'https://dl.espressif.com/dl/idf-env/idf-env.exe' -OutFile .\idf-env.exe; .\idf-env
```

Drivers are also included in the [IDF-Installer](#).

What is another way to maintain the ESP-IDF environment?[i](#)

Using [idf-env](#), you can manage several ESP-IDF versions and launchers, install drivers, manage antivirus exclusions, and more.

While trying to debug my project, I encountered an issue. What should I do?[i](#)

First, review the [ESP-IDF JTAG Debugging documentation](#) to understand how debugging works and the expected configuration of your device. The debugger connects to OpenOCD, so check the [OpenOCD Troubleshooting FAQ](#) for any OpenOCD errors you might have encountered.

If the issue is not related to OpenOCD, check your IDE integration log files and post an issue in the respective GitHub repository. Refer to the [Troubleshooting documentation](#) to identify the error in the extension.

I have an issue flashing my Espressif device. What should I do?

Start by following Espressif's [Esptool Troubleshooting documentation](#).

[Previous](#) [Next](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)

Release Notes

[ESP-IDF Extension for VSCode](#)

- »
 - Release Notes
 - [Edit on GitHub](#)
-

Release Notes

All notable changes to the Espressif IDF extension are documented in this file.

Unreleased

Features and enhancements (Unreleased)

Contribute Espressif Documentation and ESP Component Registry MCP servers to VS Code Chat. The Documentation MCP server requires a GitHub or WeChat login.

2.0.2

Replace how ESP-IDF and ESP-IDF Tools is installed and configured in the extension. The

ESP-IDF: Configure ESP-IDF extension and extension settings such as `idf.espIdfPath` and `idf.toolsPath` are removed in favor of using our new [Espressif Install Manager \(EIM\)](#). While old ESP-IDF setup should still work to ease the transition, new ESP-IDF setup are expected to use EIM. See the new [Install documentation](#) for more details.

[v2.0.2](#)

Features and enhancements (2.0.2)

[EIM integration and removal of extension settings such as `idf.espldfPath` and `idf.toolsPath` in favor of environment variables saved as extension global store state](#)

[Show templates before configuration in New Project Wizard](#)

[Debug image viewer and also view C image array from files](#)  You can configure for OpenCV, LVGL and any custom data types as long as you provide an image C `UInt8Array` and size length.

[Remove old debug adapter, ESP-MDF, ESP-Matter and ESP-HomeKit](#). Many of these frameworks are available in the ESP Component Registry. We are keeping ESP-ADF though.

Bug Fixes (2.0.2)

[Show ESP-IDF versions in descending order](#)

1.11.1

[v1.11.1](#)

Bug Fixes (1.11.1)

[Use Output Capture Execution only for Copilot Chat](#)

[Post-build always hangs in v1.11.0](#)

[Remove await for clang check in activation](#)

[Prevent Set target from closing on focus lost](#)

[Listen to restart event request to fix debug restart button](#)

[Fix get project name in IDF Size task](#)

[Add menuconfig visual separator for root sections](#)

1.11.0

[v1.11.0](#)

Features and enhancements (1.11.0)

[Add DevKits support](#)

[Add gitignore on project creation](#)

[Pre-Release branch docs](#)

[Modify event activation for file types](#)

[Add classic menuconfig in Editor Panel](#)

[Update webviews to VS Code UI Style](#)  This removes the ESP-IDF: Show Examples command in favor of ESP-IDF: New Project since it provides better customization.

[Allow customize Pytest glob pattern and unit test services](#)

[CLang install prompt if not installed](#)

[Allow additional files and directories for Full Clean commands](#)

[Extend JTAG flash arguments as configuration setting](#)

[Range support for downloads in Setup Wizard](#)

[Check OpenOCD is running before debug is launched](#)

[Add function names in Disassembly view](#)

[OpenOCD Hints in Hints Viewer](#)

[Add detect as default serial port option and use esptool.py to find serial port](#)

[Pre-release campaign notification](#)

[Prefer gdbinit prefix map with fallback to prefix map gdbinit](#)

[AI Integration with Copilot Chat using Language Tool API](#)

[Allow customize PyPi Index URL in setup wizard](#)

[Add create empty project command](#)

[Add Unity Runner and Parser, Remove Pytest](#)

Bug Fixes (1.11.0)

[Update disassemble screenshot](#)

[JTAG acronym issues](#)

[Fix IDF TARGET in multiple project configuration profiles](#)

[Fix Partial encryption in encrypted flashing](#)

[Close OpenOCD after JTAG flash end](#)

[NodeJS 20 in CI](#)

[Update build message](#)

[Fix append git and pigweed to PATH instead of prepend](#)

[Use latest in master in docs](#)

[Fix fileExists check in Setup panel](#) Thanks @jonsambro

[Use mon program esp instead of load for Symbol loading in debug](#) Thanks @wormylocks

[Move Status bar items to the left](#)

[Fix set target preview targets](#)

[Fix App trace and Heap Trace](#)

[Setup wizard misleading idf.py not found message fix](#)

[Clang and OpenOCD in PATH validation](#)

[Telemetry issues bugfixes](#)

[Fix openOCDRulesPath in addOpenOCDRules](#)

[Add constraints in pytest install step](#)

[Add double quotes around gdbinit file path](#)

1.10.1

[v1.10.1](#)

Features and enhancements (1.10.1)

[Evaluate variables on hover, view variables as Hex, set data breakpoints](#)

[Clang project settings configuration](#)

[Show currently active openOCD board in selection list](#)

[Add ESP-IDF vscode profile templates](#)

Bug Fixes (1.10.1)

[No workspace fsPath launch error](#)

[Disassembly view DAP request updates](#)

[Status bar items names on right click](#)

[Replace user HOME or USERPROFILE for HOMEPATH in doctor command logs](#)

[Add user validation before burning eFuses](#)

[Add current setup to ESP-IDF setup list](#)

[Fix OpenOCD Args in project configuration state](#)

[Execute export script in IDF Terminal, allow custom terminal executable](#)

[Add eFuse docs in readme](#)

[Remove which and where dependency, browse binary in list of PATH](#)

[Fix cloning dev branches](#)

[Add Windows ARM serialport binaries](#)

[Add idf-python and idf-git Github mirrors](#)

[Fix sysview gdbinit commands](#)

[Fix update debug adapter debugPort in launch.json](#)

1.10.0

[v1.10.0](#)

Features and enhancements (1.10.0)

[Add command to delete esp-idf specific settings](#)

[Add idf.monitorPort setting](#)

[Add idf.jtagFlashCommandExtraArgs](#)

[Use idf qemu and idf.qemuExtraArgs for debug and monitor](#)

[App bootloader partition table build flash cmds, read Partition from device, partition specific flashing](#)

Bug Fixes (1.10.0)

[Russian translation typos fix](#) Thanks @SinglWolf

[Fix esp idf.json setup not recognized](#)

[Quotes for export script in IDF Terminal](#)

[Fix preFlashTask in multiples profiles projects](#)

[Add web extension debug and other documentation](#)

[Fix openOCD arguments order](#)

[Fix project configuration documents](#)

[Fix create example project from Components registry](#)

[Use remoteName to detect Codespaces environment](#)

[Use 2 spaces for tab size on created json files](#)

[Ensure build directory exists resolve workspaceFolder paths in Project Configuration editor](#)

[Fix build flash monitor release serial port](#)

[Merge env vars and add IDF_TARGET in Project Configuration Editor and New Project Wizard](#)

Older releases

For release notes from **1.9.1** through **0.1.0**, see the [CHANGELOG on GitHub](#).

[Previous](#)

© Copyright 2016 - 2024, Espressif Systems (Shanghai) Co., Ltd.

- Built with [Sphinx](#) using a [theme](#) based on [Read the Docs Sphinx Theme](#).
- [Download PDF](#)